FixFirst Group Report

A Python Troubleshooting and Repair Verification Tool

NUS-ISS Intelligent Reasoning Systems Practice Module

October 2026

Canvas group number 24

Group members	Student ID or masked ID
1 HE ZIHANG	A0xxxx72R
2 WEI YI	A0xxxx64H
3 CHI YONGJUN	A0xxxx66A



Figure 1 From scattered errors to guided actions and checked results

Executive summary

FixFirst helps a developer or a coding agent find out why a Python project does not run, what to fix first, and when it is really fixed. It runs the project's own checks with the project's own interpreter, explains each failure with its evidence, rule and official source, proposes one concrete next step, and closes an issue only after a real check passes again. People use a local web page or the command line; coding agents use the same diagnosis through the Model Context Protocol (MCP). FixFirst never edits code or installs anything into the project's environment.

The reasoning combines 132 forward-chaining production rules, a knowledge graph of 737 removal and 12 deprecation entries from 191 sources, a session graph linking evidence, causes, actions and checks, a decision tree over 81 evidence features for cases that no rule settles, and grouping of repeated messages.

Through MCP, FixFirst raised the repair rate of the local model Gemma 4 26B-A4B from 38.9% to 90.0% on 18 sealed tasks, and its own first check of those tasks gave all 29 failures a cause class of the reference repair (17 of 17 in single-class tasks). With the failing tests named, it closed 27 of 28 unseen real code defects only after the real fix and none wrongly. Six participants were 23.4% faster in median time with it, a trend ($p = 0.094$).

Contents

1 Introduction....4
1.1 Background and motivation....4
1.2 Existing tools and literature....4
1.3 What is new....6
2 Problem Analysis and Requirements....7
2.1 Project objectives....7
2.2 Pain points....7
2.3 Functional requirements....8
2.4 Non-functional requirements....9
3 Use Case Analysis....10
3.1 Actors and use cases....10
3.2 Use case 1: check a failing project and verify the fix....10
3.3 Use case 2: a coding agent repairs the project through MCP....11
3.4 Use case 3: export a shareable report....12
4 System Architecture and Design....13
4.1 Architecture overview....13
4.2 Component description....14
4.3 Data flow....15
4.4 Core workflow....16
4.5 Evidence and cause classes....16
4.6 Scope....17
5 Machine Learning and Reasoning Design....18
5.1 Knowledge rules guide the next step....18
5.2 A knowledge graph explains the evidence....19
5.3 Data mining reduces repeated information....19
5.4 The decision tree....19
5.5 Design choices and their evidence....20
5.6 Models and baselines compared....21
6 System Implementation....22
6.1 Technology stack....22
6.2 Front end and back end....22
6.3 Data storage....22
6.4 The product in use....23
6.5 Engineering process and versions....25
6.6 Safety and limits....25
6.7 Windows....25
7 Evaluation and Results....27
7.1 Overview....27
7.2 Machine learning evaluation....28
7.3 System evaluation....32
7.4 User evaluation....35

8 Discussion and Limitations..... 36
8.1 What the results mean..... 36
8.2 Objectives achieved and not achieved..... 36
8.3 Comparison with the baselines..... 37
8.4 Limitations..... 38
8.5 Future work..... 38
9 Team Roles and Contributions..... 39
References..... 40
AI assistance statement..... 41
Appendix A Project proposal..... 42
Appendix B Mapping to the MR, RS and CGS courses..... 53
Appendix C Installation and user guide..... 54
Appendix D Experiment details..... 62

1 Introduction

1.1 Background and motivation

A repository can be available while its execution environment is no longer reproducible. An import failure can mean a missing distribution, a local path problem, a standard library name that was removed, or a library version that no longer offers an API. Installing a package named after the import is therefore not always a useful first action, and a passing check of the installed dependency metadata[1] does not show that the application runs. Several such faults often occur together, and repairing one can uncover the next: a failing test stays hidden behind a collection error until an import is repaired. A developer then reads long tracebacks, searches for each message and tries changes one at a time, without a record of what has already been checked.

The problem is common in research data. Mukherjee, Almanza and Rubio-González examined 2,702 historical Python builds; their PyDFix study found dependency-related reproducibility failures in 1,921 and obtained complete fixes for 859[2]. Peng et al. found potential configuration issues in 183,864 releases of PyPI libraries, 68% of which only a source-level check could detect[3]. These figures come from those studies' datasets, not from a failure rate of all Python projects; our own early investigation of thirteen projects found the same pattern. The cost is time and trust: a newcomer may spend an afternoon on an environment before reading any of the code, and a fix that only seems to work can return in a teammate's environment.

AI assistants now take part in this work: 84% of the respondents to the 2025 Stack Overflow developer survey use or plan to use AI tools in their development process[4]. Their answers are not always reliable. Asked about their frustrations with AI tools, 66% of those who answered named "AI solutions that are almost right, but not quite", and more respondents distrust the accuracy of AI tools (46%) than trust it (33%)[4]. An answer that sounds right is not yet a repair that has been checked, and for a coding agent a wrong answer costs wasted steps or a repair that passes by editing the tests.

FixFirst addresses the whole loop. Given a Python project, the interpreter it should run with and a goal such as making its tests pass, it shows which failure blocks the goal, why it happens, what to do first, and whether a change has really removed it. Its users are students running an unfamiliar assignment, developers taking over a repository, maintainers helping new contributors, researchers reproducing published code, and the coding agents that assist them. It covers the recurring causes described in Section 4.5 and leaves application logic, which needs human judgement, to the developer.

1.2 Existing tools and literature

Table 1 compares the workflows of related tools. It does not claim that these tools cannot diagnose errors, or that FixFirst replaces a coding agent.

Table 1 Related tools and their relationship to FixFirst.

Tool or approach	Main role	Relationship to FixFirst
pip check	Checks the installed dependency metadata[1]	Useful evidence; does not run the project's program or tests
pipdeptree	Shows installed dependency relationships[5]	Helps inspect an environment; FixFirst links such facts to observed failures
Dependabot, Renovate	Propose and automate configured dependency updates[6,7]	Maintenance from the outside; FixFirst starts from the failure in the current environment
tox	Runs checks in configured environments[8]	Reproduces checks; FixFirst explains the evidence and orders the actions
AI assistants in the IDE, e.g. GitHub Copilot agent mode	Explain errors, edit code, run commands and iterate[9]	Complementary: through MCP, FixFirst gives such an agent an evidence-backed next step and a check that verifies it
Error monitoring with AI debugging, e.g. Sentry Seer	Root causes and fixes from runtime signals such as error events, traces and logs[10]	Needs an instrumented application that runs; FixFirst starts from a local project whose checks fail

PyDFix analyses dependency-related build failures and searches for version changes that restore reproducibility[2]. FixFirst shares the use of real failure evidence and of verification afterwards; its advice also covers local modules and project configuration, and the change is made by a person or an agent. DockerizeMe infers the dependencies of Python code snippets from package knowledge with graph-based inference[11], and motivates our mapping from imported modules to distributions; FixFirst additionally checks whether an import refers to the standard library or to a project file before it suggests an installation. PyConf detects configuration issues of PyPI libraries in their setup, packing and usage stages from the source[3]; FixFirst reads a project's declarations statically for the same reason, but starts from an observed failure. BugsInPy provides real Python defects with their tests[12]: 493 bugs in 17 projects in the paper, 501 in the same 17 projects in the index we used (commit 11c5f1e; the eight extra are all in black). We use it to test that a diagnostic tool does not blame the environment for a code defect.

The repair percentages these studies report are not comparable with our agent experiment: tasks, interventions and success criteria differ.

Business case. The submitted product is a free community edition with local web, command line and MCP interfaces; no account is needed. A client that uses a commercial language model has its own provider charges, separate from FixFirst. A later paid offer could be maintenance and integration support, shared team rules or continuous-integration reporting. These are commercial hypotheses: we have no customer survey from which to estimate the market size or the willingness to pay.

1.3 What is new

Three things set FixFirst apart.

1. **A fix counts only when a real check passes.** Every issue stays open until the project's own test or program runs successfully again with the same interpreter and scope; editing the tests does not count.
2. **One diagnosis for people and agents.** The same evidence, rules and plan are shown in a web page and returned to a coding agent through MCP, so an agent receives an evidence-backed next step and a way to verify it.
3. **Advice that knows what not to do.** Before suggesting an installation, FixFirst checks whether the name is a standard library module, a project file or a removed API, and it does not offer a version change that the project's own declarations forbid.

2 Problem Analysis and Requirements

This chapter restates the objectives of the proposal, maps the pain points it identified to the features FixFirst delivers, and turns them into requirements.

2.1 Project objectives

The proposal set three goals: reduce the time spent reading repeated errors, make recommendations understandable, and verify progress with real check results. Table 2 refines them into five objectives with the measures the proposal set for each; Section 8.2 compares them with the results.

Table 2 Objectives and the measures set in the proposal.

Objective	Measure set in the proposal
O1 Bring scattered check results into one place and reduce repeated messages	Message grouping with a precision of 0.90 on hard cases
O2 Recommend a useful first step for the selected goal	Next-step guidance better than naive orderings
O3 Make every recommendation understandable and checkable	A participant study with 6–8 participants, at least 80% completion and a median time 20% lower
O4 Count progress only from real check results	Repairs verified on at least 8 real defects in at least 4 libraries
O5 Give coding agents the same diagnosis	A comparison of agents with and without FixFirst

2.2 Pain points

The proposal identified six pain points. Table 3 maps each of them, and one more found during the project, to the feature that FixFirst now delivers and to the benefit for the user. Sections 2.3 and 2.4 turn them into functional and non-functional requirements.

Table 3 Pain points, the features that address them and the benefit for the user.

Pain point	Feature	User benefit
Results are scattered.	Runs the project’s own tests or program with its own interpreter, reads its declarations and installed distributions, and keeps all results in one session.	One place to see what failed and why.
The same error appears repeatedly.	Groups repeated messages, first by structure and then by TF-IDF similarity; the original messages stay visible.	Fewer messages to read.
The next step is unclear.	Orders the actions by the goal, their preconditions, their impact, the evidence and their cost; the first step is a concrete action.	A clear starting point.
Advice is hard to trust.	Shows the error, the rule and the official source behind each step, and how the repair will be confirmed.	Advice that can be checked.

Pain point	Feature	User benefit
The effect of a change is uncertain.	*Check again* closes an issue only when a real check passes with the same interpreter and scope; changed tests are reported.	Clear fixed, open and unchecked states.
The process is hard to explain.	Exports a shareable HTML or JSON report with the home directory redacted; every check is recorded.	Easier help and handover.
Coding agents guess and do not verify.	Offers the same diagnosis through MCP with the tools `diagnose`, `check_again` and `explain`.	An evidence-backed step for the agent and a way to verify it.

2.3 Functional requirements

Table 4 Functional requirements and how FixFirst meets them.

ID	Requirement	How FixFirst meets it
FR1	Accept a project, its interpreter, a goal and, optionally, the tests to check.	The web form, the command line and the MCP tool `diagnose` take the same inputs; the web form finds the project's environment, for example `.venv`, and fills in its interpreter.
FR2	Run the relevant checks with the project's own interpreter.	Check runners for pytest with a probe plugin, unittest, a program, notebooks, `pip check` and Ruff.
FR3	Turn check output into issues with evidence.	Parsers create events, repeated messages are grouped, and each issue records its evidence: exception, location, module status and 81 features.
FR4	Determine the cause of each issue.	Production rules use the domain knowledge graph; when no rule settles the cause of a test failure, a decision tree suggests a possible cause, shown as not confirmed.
FR5	Propose an ordered plan of next steps.	The planner orders actions by preconditions, impact on the goal, evidence and cost; version advice respects the project's declarations.
FR6	Explain every step.	Each step shows the error, the rule, the official source and how it will be confirmed; `explain` returns the same through MCP.
FR7	Verify a change.	*Check again* closes an issue only after a passing real check with the same interpreter and scope; edits to protected tests and changes of interpreter are reported.
FR8	Share the result.	A shareable HTML or JSON report with the home directory redacted; saved sessions can be reopened.
FR9	Serve coding agents.	An MCP server over standard input and output with the tools `diagnose`, `check_again` and `explain`.
FR10	Look for a working release when asked.	An optional release search installs at most 12 releases on trial in an isolated environment.

2.4 Non-functional requirements

Table 5 Non-functional requirements, the design response and the evidence.

Quality	Requirement	Design response and evidence
Safety	Never change the project or its environment without the user.	FixFirst does not edit code or install packages; release trials run in an isolated environment (Section 6.6).
Trustworthiness	Never report a fix that a real check has not confirmed.	Verification is bound to the goal, the interpreter and the scope; no issue was closed wrongly on 75 held-out real defects (Section 7.3.2).
Explainability	Make every conclusion traceable to evidence and a source.	Every rule conclusion keeps its facts and its source; 191 source records; a session graph links all steps (Chapter 5).
Privacy	Keep project data on the user’s machine.	The web page is served on 127.0.0.1 with a token for each launch; no account; exported reports redact the home directory.
Portability	Run on macOS, Linux and Windows with Python 3.10 or later.	Continuous integration on Ubuntu and Windows; development and all experiments on macOS; a manual check of 0.8.0rc5 and an interactive check of the released 0.8.0 passed on Windows 11; the release corrects the one Windows defect that the full test suite found (Section 6.7).
Usability	Be easy to learn for developers.	A mean System Usability Scale score of 79.6 from six participants (Section 7.4).
Maintainability	Detect regressions automatically.	1,151 test functions in 98 modules and Ruff run on Ubuntu and Windows for pushes to main and for every pull request (Section 6.1).
Interoperability	Work with any MCP client.	JSON-RPC over standard input and output, accepting the MCP protocol versions 2025-11-25, 2025-06-18, 2025-03-26 and 2024-11-05[13].

3 Use Case Analysis

3.1 Actors and use cases

Two primary actors use FixFirst (Figure 2). A developer works through the local web page or the command line; a coding agent, such as an assistant in an IDE or an autonomous agent, calls the same diagnosis through MCP. Two secondary actors are systems: the project, whose interpreter, tests and files FixFirst runs and reads, and the Python package index, which only the optional release search contacts. Through MCP, the agent's use case includes checking the project and checking it again; reading the evidence for a step, the MCP tool `explain`, extends it when the agent asks for it, as naming the tests extends a check.

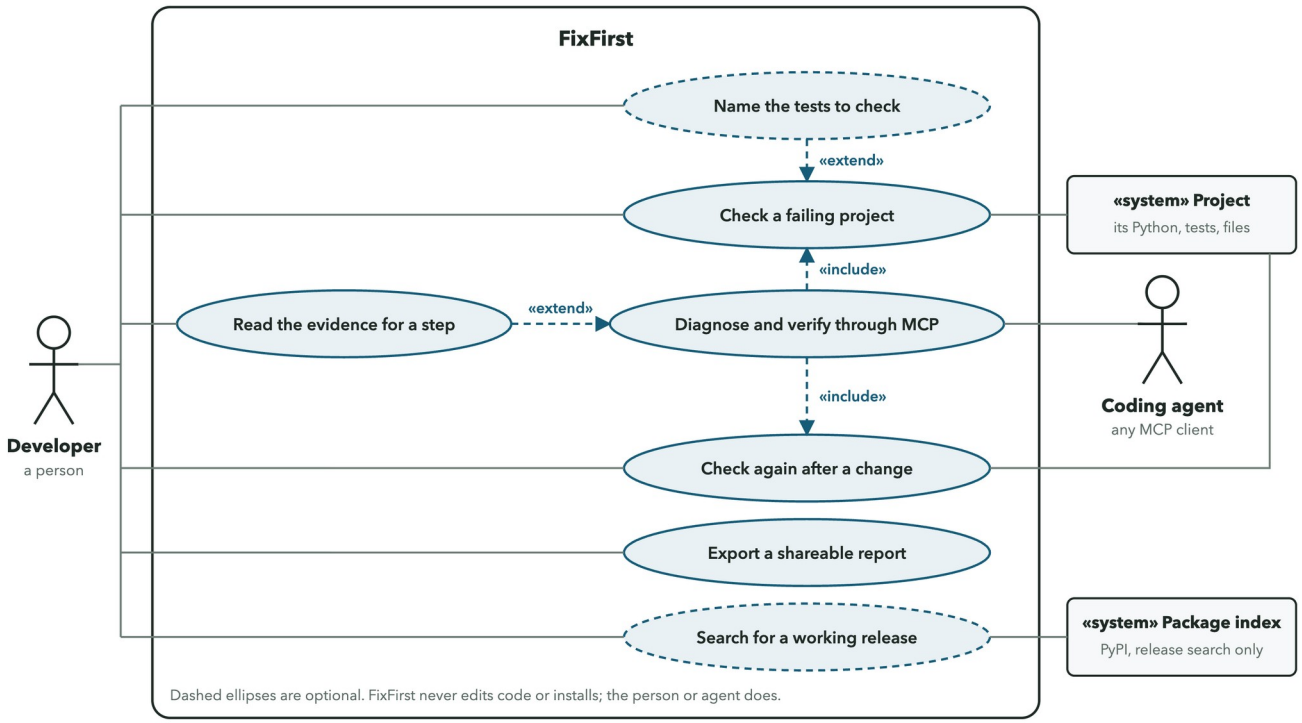


Figure 2 Use case diagram. The developer and the coding agent are the primary actors; the project and the package index are systems that FixFirst uses.

The three core use cases follow. Use case 1 is the main loop for a person, use case 2 the same loop for a coding agent, and use case 3 hands the result to someone else. The remaining use cases in Figure 2 are steps within them.

3.2 Use case 1: check a failing project and verify the fix

Table 6 Use case 1.

Item	Description
Actors	Developer (primary); the project (secondary).
Goal	Make the project's tests pass, starting from a project whose tests fail or do not start.

Item	Description
Preconditions	FixFirst is installed and started; the project and its Python environment are on the developer's computer; the developer trusts the project's code, because checks run it.
Main flow	1. The developer enters the project folder; FixFirst finds the project's interpreter, its test files and pytest, and shows them for confirmation. 2. The developer chooses the goal *Make my tests pass* and may name the tests to check. 3. FixFirst runs the tests with the project's interpreter, groups the output into issues and shows ordered steps, for example "2 problems to fix". 4. The developer opens the first step and reads the error, the rule and the official source behind it, and how the repair will be confirmed. 5. The developer makes the change in an editor. 6. The developer selects *Check again*: FixFirst runs the same tests with the same interpreter, closes the issues whose checks now pass and shows failures that were hidden before. 7. Steps 4 to 6 repeat until the goal is reached.
Alternative flows	Only a heuristic or the decision tree supports the cause: the step says *Likely* or *Possible cause* and *not confirmed*, and asks for an inspection first. The change edits a protected test or the interpreter changes: FixFirst reports it and does not count the issue as fixed. The fault is in the project's own code: FixFirst names a code or test defect and gives no installation command. The developer may ask for a release search.
Result	All tests pass, verified by a real test run; each issue is listed as fixed and verified; the session records every check, plan and verdict.

In the demonstration project of our videos, the first check finds two import errors: `collections.Mapping`, which Python 3.10 removed, and an import of `helper` although the project contains `helpers.py`. After the two edits, *Check again* confirms both and reveals two failures that were hidden behind them, `numpy.float` and a missing configuration value. After two more edits all four issues are fixed and verified.

3.3 Use case 2: a coding agent repairs the project through MCP

Table 7 Use case 2.

Item	Description
Actors	Coding agent (primary); the developer, who supervises; the project (secondary).
Goal	Repair the project so that its tests pass, with every step verified.
Preconditions	The agent's client has FixFirst's MCP server configured; the agent can read and edit files and run commands; the developer approves tool calls as the client requires.
Main flow	1. The agent calls `diagnose` with the project, the interpreter and the goal. 2. FixFirst returns a report that puts the action first, followed by the place, the cause, the rule and how to confirm it (Figure 7). 3. The agent edits the file named in the first step. 4. The agent calls `check_again`; FixFirst runs the checks and reports which issues are fixed and which new ones appeared. 5. Steps 3 and 4 repeat; the agent may call `explain` for the evidence of a step. 6. The loop ends when the report states that the goal is reached and verified by a real check.

Item	Description
Alternative flows	The agent edits a test to make it pass: the protection rule reports the changed tests and the issue stays open. The agent does not follow the advice: verification still requires a real passing check.
Result	The tests pass with the project’s interpreter; the last report reads “All tests pass. Verified by test run”.

3.4 Use case 3: export a shareable report

Table 8 Use case 3.

Item	Description
Actors	Developer.
Goal	Hand the state of the project to a teammate or a helper.
Preconditions	A session with at least one check exists.
Main flow	1. The developer selects *Download a shareable report* in the web page, or runs `fixfirst export` on the command line. 2. FixFirst writes an HTML or JSON report with the environment, the checks that ran, the issues fixed and still open, and their evidence, with the home directory redacted. 3. The developer shares the file.
Result	The recipient can see what was checked and what remains without access to the developer’s computer.

4 System Architecture and Design

4.1 Architecture overview

FixFirst is built in layers (Figure 3). A web interface, a command line and an MCP server for coding agents all call one session service, which holds the project, its interpreter, the goal and the tests to run, and records every check, plan and verdict. The observation layer runs the checks and reads the project's files and environment; the interpretation layer turns their output into grouped issues and evidence; the reasoning layer combines the knowledge graph, the rules and the decision tree, orders the actions and decides what a later check has verified. A session graph keeps the links between all of these, so that every step can be explained.

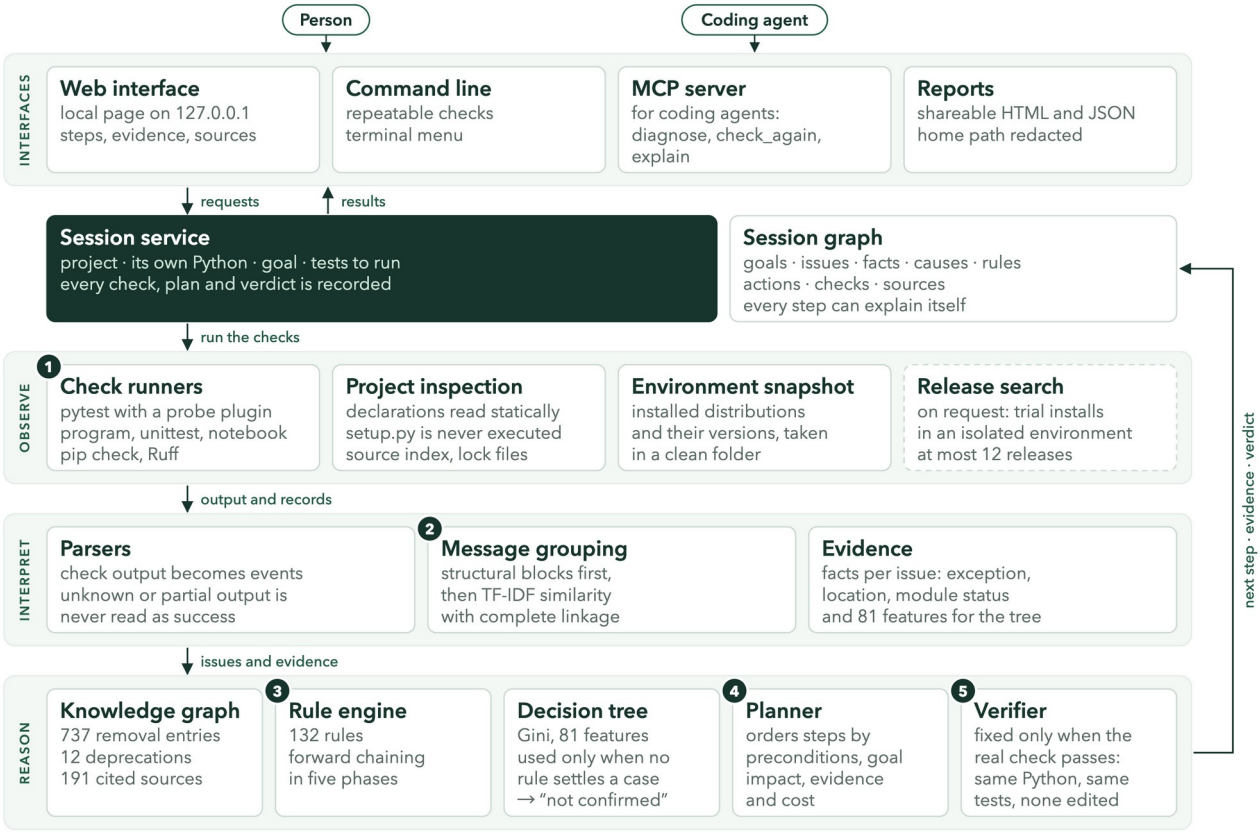


Figure 3 Architecture. Three interfaces share one session service; the checks are observed, interpreted and reasoned over in layers, and the result returns to the session. The numbers mark where the five steps of the diagnostic loop in Figure 5 take place.

Why this architecture. Five decisions shaped it.

- *One session service behind all interfaces.* A person and an agent see the same diagnosis, and each interface stays thin.
- *Separate layers for observation, interpretation and reasoning.* Saved observations can be replayed without running the project again, which makes the reasoning testable and most of the evaluations in Chapter 7 repeatable.
- *Knowledge first, learning as a fallback.* Rules and the knowledge graph give precise conclusions that can be explained; the decision tree generalises, but its answers are shown as a possible cause, not confirmed.

- *Provenance everywhere.* The session graph records which facts, rules and sources led to each step.
- *Observe and advise, never act.* Verification by real checks means something only if the tool does not change what it verifies.

4.2 Component description

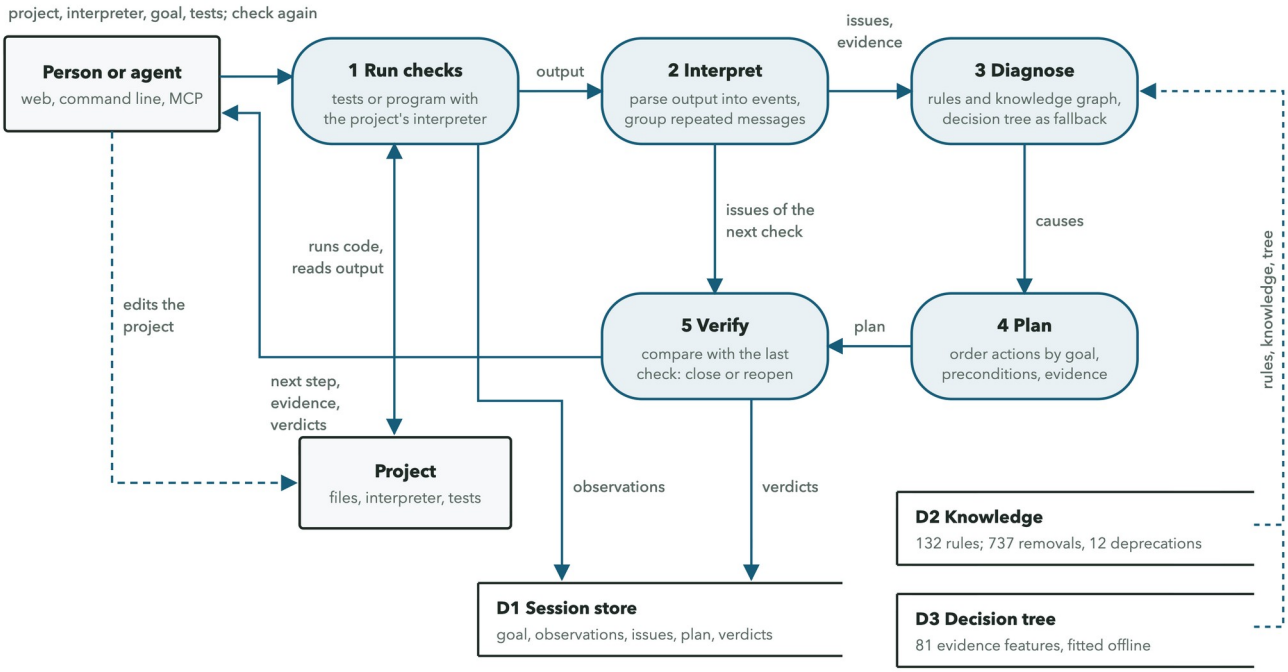
Table 9 Components, their responsibilities and their main modules in the package.

Layer	Component	Responsibility	Main modules
Interfaces	Web interface	Local page on 127.0.0.1: project, goal, ordered steps with evidence and sources, *Check again*, report download	`web.py`, `workspace.py`, `templates/`
	Command line	Repeatable checks, a terminal menu and report export	`cli.py`, `interactive.py`
	MCP server	`diagnose`, `check_again` and `explain` for coding agents over standard input and output	`mcp_server.py`, `facts.py`
	Reports	Shareable HTML and JSON with the home directory redacted	`report.py`
Session	Session service	Holds the project, the interpreter, the goal and the tests to run; records every check, plan and verdict	`service.py`, `storage.py`
	Session graph	Links goals, issues, facts, causes, rules, actions, checks and sources	`knowledge_graph.py`
Observe	Check runners	Run pytest with a probe plugin, unittest, a program, notebooks, `pip check` and Ruff	`runner.py`, `execution.py`
	Project inspection	Reads declarations statically (`setup.py` is never executed), the source index and lock files	`project.py`, `dependency_context.py`
	Environment snapshot	Installed distributions and their versions, taken in a clean folder	`probe.py`
	Release search	On request, trial installs of at most 12 releases in an isolated environment	`versions.py`
Interpret	Parsers	Turn check output into events; unknown or partial output is never read as success	`parsers.py`, `execution_parsers.py`
	Message grouping	Structural blocks first, then TF-IDF similarity with complete linkage	`grouping.py`
	Evidence	Facts for each issue, and the 81 features of the decision tree	`evidence.py`, `symbol_context.py`, `source_context.py`
Reason	Knowledge graph	Removed and deprecated APIs with their sources	`domain.py`, `knowledge/domain.toml`

Layer	Component	Responsibility	Main modules
	Rule engine	Forward chaining over production rules in five phases	`engine.py`, `knowledge/rules.toml`
	Decision tree	A possible cause, shown as not confirmed, when no rule settles a test failure	`classification.py`, `knowledge/diagnosis_tree.json`
	Planner	Orders the steps by preconditions, goal impact, evidence and cost	`reasoning.py`
	Verifier	Closes an issue only when the real check passes with the same Python and the same tests, none edited	`service.py`, `integrity.py`

4.3 Data flow

Figure 4 shows how data moves between the person or agent, the project and the five processes of the diagnostic loop. The requests carry the project, the interpreter, the goal and the tests; the replies carry the next step, its evidence and the verdicts. Process 1 runs the project's code and reads its output; processes 2 and 3 turn the output into issues and causes, reading the rules, the knowledge graph and the decision tree; process 4 orders the actions; process 5 compares the issues of the next check with the plan, and closes or reopens them. The session store keeps the observations and the verdicts, so that a session can be reopened, replayed and explained. The only arrow into the project that changes it comes from the person or the agent.



Processes are numbered in the order of the diagnostic loop; dashed lines are edits by the person or agent and reads of knowledge.

Figure 4 Data flow. Processes, data stores and the external entities they exchange data with.

4.4 Core workflow

The user chooses a project, its Python interpreter and a goal, for example *make my tests pass*. FixFirst runs the relevant checks, groups related findings, derives evidence facts, consults domain knowledge, applies rules, orders the actions and records how the next step will be verified (Figure 5). Fixing one problem can uncover the next one, for instance a test failure that was hidden behind a collection error, so the plan is computed again after every check rather than fixed in advance.

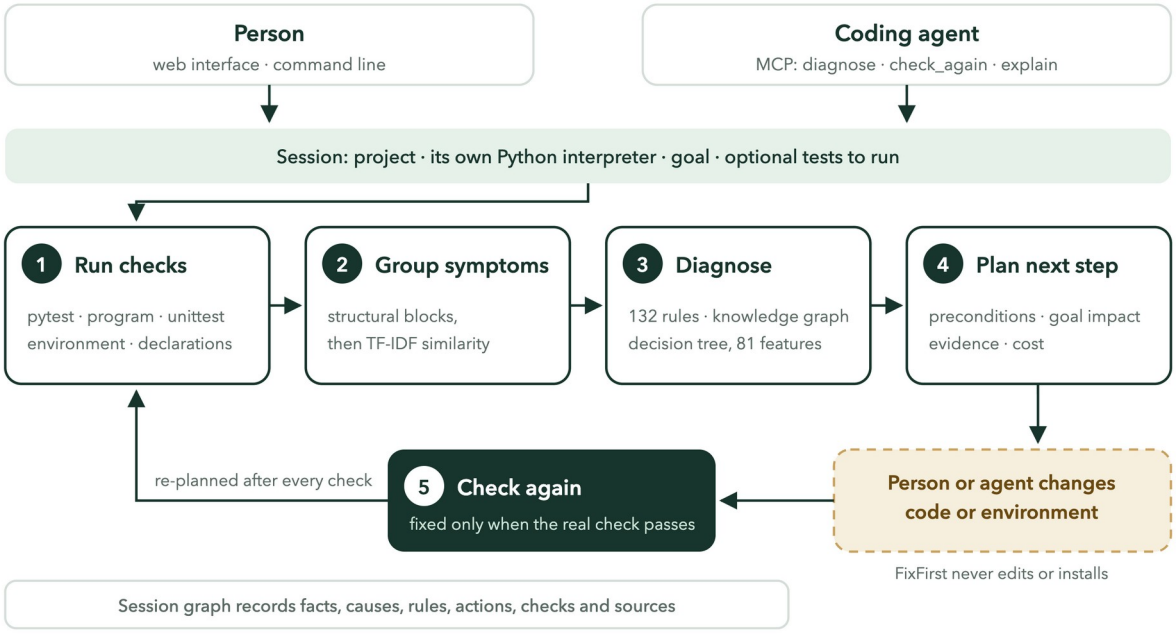


Figure 5 The diagnostic loop. FixFirst runs the checks, diagnoses, plans and verifies; the person or agent makes every change.

4.4.1 Turn check results into useful actions

Actions are ordered by their preconditions, their impact on the goal, the strength of the evidence and their cost. The first action is meant to be something the user can do, or an explanation of a constraint that needs review. Version suggestions must respect the project's declarations: if a declaration excludes the compatible range, no installation command is offered as a repair.

4.4.2 Use new results to update the plan

Verification is bound to the goal, the interpreter and the scope of the check. A completed passing test run differs from a cancelled, partial or collection-only run; for a script, success applies to the saved entry, arguments and input. Since 0.8.0rc3 the user or agent can name the tests to check, and the selection is reused on every later check. Since 0.8.0rc5, when a project sets its own pytest options, a repair is confirmed by a second run with those options. After every check the plan is computed again from the new evidence.

4.5 Evidence and cause classes

FixFirst distinguishes five cause classes: a **missing dependency**, a **local module or import path** problem, a **version incompatibility**, **missing configuration**, and a **defect in the project's code**

or tests. The last is a boundary: an assertion failure may need an investigation of the application logic, which FixFirst does not attempt to repair.

Evidence includes the exception and its location, installed package metadata, the project's static declarations, import relationships and the outcome of the selected program or tests. Where supported, the runtime identity of a receiver tells a real library object from a project class with the same name. Declarations are read statically (`setup.py` is never executed), and a lock file is treated as a recorded choice, not as proof that the version works. When evidence is missing or exceeds an inspection limit, the system stays conservative and says so.

4.6 Scope

In scope. Python projects whose goal can be checked by running their tests with pytest or unittest, or a program; the first four cause classes, which FixFirst diagnoses and for which it proposes repairs; code and test defects, which it recognises as such without repairing them; use through the web page, the command line and MCP on macOS, Linux and Windows.

Out of scope. Repairing application logic; editing code or installing packages automatically; projects in other languages; running checks on remote machines.

5 Machine Learning and Reasoning Design

FixFirst's reasoning is a hybrid. Knowledge-based components decide first: production rules over a domain knowledge graph derive causes and actions with their sources. A learned decision tree suggests a possible cause only where no rule settles a test failure, and one data-mining step reduces repeated messages. Table 10 relates the parts to the technique groups of the course; Appendix B maps them to the MR, RS and CGS courses.

Table 10 Course technique groups, the methods FixFirst uses and their role in the product.

Course technique group	Methods	Product role
Decision automation and knowledge-based reasoning	Forward chaining over 132 production rules in five phases; action rules with preconditions; goal-based ordering	Derive causes and feasible next actions from known facts and rank them for the selected goal
Knowledge discovery and data mining	Character n-gram TF-IDF and cosine similarity with complete linkage inside structural blocks; a Gini decision tree over 81 evidence features	Reduce repeated messages; suggest a possible cause when no rule settles a test failure
Cognitive techniques and knowledge representation	A domain knowledge graph of removed and deprecated APIs with their sources; a session evidence graph with breadth-first traversal; questions mapped to graph queries	Link issues, recommendations and evidence to explain what to do and what remains unchecked
Resource optimisation (partial)	A bounded search over compatible releases with a trial budget	Find a release that passes the tested import; not presented as A* or evolutionary search

5.1 Knowledge rules guide the next step

The rule engine is a forward-chaining production system with stratified negation. The 132 rules are kept in a TOML file and run in five phases, each until no new fact appears: derive (15 rules), diagnose (39), heuristic (12), fallback (1) and plan (65). Rule conditions match facts with variables, negation and tests such as version comparisons, and every derived fact keeps the facts, the rule and the source that produced it.

For example, an import of `collections.Mapping` on Python 3.12 is related to the removal documented for Python 3.10; the proposed edit is to import `Mapping` from `collections.abc`; and the issue is closed only after a relevant check passes. A cause that only a heuristic rule supports is shown as *Likely … · not confirmed*.

The planner orders the actions by six keys in turn: actions whose preconditions are unmet come last; then a higher impact on the goal (the issues an action affects or unblocks); then stronger evidence (knowledge before other reasons, hypotheses last); then the kind of action (inspect before a manual fix before a re-run); then a lower cost; and finally a stable identifier, so that the same evidence always gives the same plan.

5.2 A knowledge graph explains the evidence

The domain knowledge file holds 737 removal entries (439 for APIs, 152 for usages, 68 for keyword arguments, 58 for modules, 19 for attributes and 1 for a fixture), which together name 1,816 removed names; 12 deprecation entries (35 names); and 191 source records, each with a URL to the upstream release notes or documentation. Smaller tables relate import names to distributions (42 mappings) and describe test fixtures (21 entries), exceptions and tool failures. These are counts of curated entries, not of independent libraries or of verified replacement programs; some entries were checked by running them with scripts in the repository. The command `fixfirst knowledge` exports the knowledge as a graph with 11 node types and 10 relation types.

A second graph is built for each session. It links goals, issues, facts, causes, rules, actions, checks and sources with 10 node types and 15 typed relations. A question such as “why is this the first step?” is matched to an intent, in English or Chinese, and answered by a breadth-first traversal of the graph (at most 120 nodes, depth 10) or by a lookup in the domain knowledge; no language model is involved.

5.3 Data mining reduces repeated information

Messages are first cleaned: colour codes are removed, and timestamps, temporary paths and object addresses are replaced by placeholders. Grouping then works in two steps. Events are placed in structural blocks: the same tool, stage, kind, component and version tokens, and for Ruff the same location. Within a block, messages are compared by TF-IDF over character n-grams of three to five characters and cosine similarity, and joined by complete linkage at a threshold of 0.82, so that two messages are never chained together through a third. Each issue keeps its member events and evidence, the lowest pairwise similarity is kept as its group score, and the original events stay in the session. A group is a set of similar messages, not a root cause. Exact matching, and a local sentence-embedding model as an optional extra, can replace TF-IDF.

5.4 The decision tree

Data collection. A case generator combines five small project templates with failure scenarios; each case is created as a real project with a real fault, and its checks are actually run. The label comes from the scenario definition. The training set contains 215 such cases, five templates times 43 scenarios: 65 version incompatibilities, 60 code defects, and 30 each of missing dependencies, local module problems and missing configuration. A 44th scenario was added later; it appears in the 220 evaluation cases (Section 7.2.1) but not in the training set.

Preprocessing. The output of each check passes through the parsers into events and issues, and an evidence record is computed for every issue. Labels and the parsers’ own categories are excluded from the features, so that the tree cannot copy the parser’s answer.

Feature engineering. The 81 features fall into six groups: 44 original evidence features (exception type, stage, where the error was raised, the share of third-party frames, module status and other signals), 5 context features, 12 features from the source context, 4 from the history of the interface, 15 symbol features and 1 feature for the history of a qualified attribute.

Model and hyperparameters. The model is a CART classification tree fitted with scikit-learn[14] (Table 11). The shipped tree has 13 nodes and 7 leaves, a depth of 5, and uses 6 of the 81 features; the most important are whether the failing code uses an API of an installed or standard-library module (importance 0.31), a configuration context (0.20), whether a module is mentioned (0.19) and project names in the context (0.16). Every leaf holds training cases of a single class, so every prediction of the shipped tree has a confidence of 1.0; this is not a calibrated probability.

Table 11 Hyperparameters of the decision tree.

Hyperparameter	Value	Reason
Split criterion	Gini impurity	Standard for CART classification; fast on small tabular data
Maximum depth	6	Keeps the tree small enough to read in full and limits overfitting on 215 cases
Minimum samples per leaf	2	No leaf rests on a single case
Random seed	42	The shipped tree can be rebuilt byte for byte
Sample weight	1.0 for every case	All scenarios count equally

Training process. `fixfirst dataset --suite diagnosis` generates the cases and `fixfirst evaluate` evaluates the method with leave-one-template-out and leave-one-scenario-out cross-validation, fitting a separate tree for each fold. The shipped model is stored as portable JSON, with a readable text export; nothing is unpickled at runtime. A script in the repository rebuilds it byte for byte with scikit-learn 1.9.1.

Inference pipeline. The product loads the tree from the package and never retrains it. It consults the tree only for issues of pytest checks, because the tree was evaluated on such cases. A prediction is kept only with a confidence of at least 0.6 and when no evidence guard forbids it, for example when the inspection of a receiver exceeded its limit; with the shipped tree the threshold never applies, so the guards decide. It then enters the session as a hypothesis. If no rule has diagnosed the issue and no heuristic has named a likely cause, a fallback rule turns the hypothesis into a suspected cause, and a plan rule proposes an inspection step: a lead to check, not a conclusion. The web page shows it as *Possible cause · not confirmed* and the MCP report as “Possible cause, not confirmed (decision tree)”. Options of the command line choose another model or turn the tree off.

5.5 Design choices and their evidence

Each main choice rests on a result of Chapter 7.

- **Rules and knowledge first.** They conclude only what they can support and can encode what not to do, such as never offering an installation for a standard library module. They were right in all 183 generated cases they answered (Table 18) and gave a class of the reference repair for all 29 failures of the unseen sealed tasks; without the knowledge graph the system fell from 29 to 20 (Table 19).

- **A small tree as the fallback.** It covers evidence patterns that no rule settles and can be read in full: alone it named the right class in 191 of 220 cases with a scenario held out and for 23 of 29 failures of the sealed tasks.
- **No language model inside the product.** The same evidence must give the same plan, offline and without an account. As baselines, three local models came close to FixFirst on the ordinary cases but recognised only 15 to 18 of the 30 difficult ones (Table 20).

5.6 Models and baselines compared

Table 12 lists every variant of FixFirst and every baseline in the evaluation. The local models ran through omlx 0.6.4 on an Apple M5 Pro with 48 GB of memory.

Table 12 Variants of FixFirst and the baselines they are compared with.

Model or baseline	What it is	Evaluated in
Parser category only	The cause class implied by the parser that read the output	7.2.1
Rules without the knowledge graph	The production rules with the domain knowledge removed	7.2.1
Rules, knowledge graph and heuristics	The knowledge-based part alone, without the tree	7.2.1
Decision tree only	The tree on every case	7.2.1
Rules and heuristics without the knowledge graph, then the tree	Knowledge removed, tree as fallback	7.2.1
Full hybrid	Rules, knowledge graph and heuristics, then the tree: the product	7.2.1
Qwen3.6-35B-A3B (6-bit)[15]	Local language model, mixture of experts with about 3B active parameters	7.2.3, 7.3.1
Gemma 4 26B-A4B (4-bit QAT)[16]	Local language model, mixture of experts with about 4B active parameters, quantisation-aware trained	7.2.3, 7.3.1
Qwen3.8-27B (4-bit)[17]	Local dense language model	7.2.3, 7.3.1
DeepSeek V4.1 Flash[18]	Cloud language model, as a coding agent	7.3.1
Agents without FixFirst	The same models with general file and command tools only	7.3.1
Advice-following script	Executes FixFirst’s printed advice, without a language model	7.3.3
Naive orderings	The order of the messages; a fixed order of cause categories	7.2.5
Exact matching	Groups only identical messages	7.2.4

6 System Implementation

6.1 Technology stack

Table 13 Technology stack.

Part	Technology
Language and packaging	Python 3.10 or later; an installable package with the command `fixfirst` (also `python -m fixfirst`)
Data models and storage	pydantic models saved as JSON files; no database
Web interface	The standard library's threaded HTTP server with Jinja2 templates, bound to 127.0.0.1 on a free port; every request carries a token created for each launch, and the Host header is checked
Command line	About two dozen subcommands, for example `scan` to run the checks, `show`, `run` for a check from the plan, `report` and `export`, `interactive` for the terminal menu, `serve` for the web page and `mcp` for agents
MCP server	JSON-RPC over standard input and output without further dependencies; protocol versions 2025-11-25, 2025-06-18, 2025-03-26 and 2024-11-05[13]
Knowledge	TOML files: the 132 rules and the domain knowledge with its 191 sources
Machine learning	scikit-learn fits the decision tree offline; the product reads the exported JSON tree with its own code
Other libraries	packaging for version comparison, PyYAML, Jinja2; optional extras for development, notebooks and sentence embeddings
Tests and quality	1,151 test functions in 98 modules with pytest[19], and Ruff[20]; one GitHub Actions workflow runs both on Ubuntu and on Windows with Python 3.12 for pushes to main and for every pull request

6.2 Front end and back end

The web page and the local server exchange JSON over HTTP on 127.0.0.1; the server rejects requests without the launch token. The server calls the same session service as the command line and the MCP server, so the three interfaces cannot disagree. The page is built from a view of the session: the ordered steps with their tags (*Start here*, the cause, or *Possible cause · not confirmed*), the error, the rule, the source and how the repair will be confirmed. *Check again* starts a new run with the same interpreter, goal and tests and redraws the view; *Download a shareable report* exports the session.

The MCP server answers `diagnose`, `check_again` and `explain` with text that puts the action first (Figure 7). A separate mode, built for the agent experiment, offers a single tool, `observe`, that returns only what the checks showed, without a diagnosis or advice.

6.3 Data storage

FixFirst needs no database. Each session is one JSON document, `session-<id>/session.json`, written atomically under an advisory lock. The store is chosen by an option, an environment

variable or a `.fixfirst` folder in the working directory; the MCP server uses `~/.fixfirst/sessions`. Table 14 lists the main parts of a session.

Table 14 Main parts of a saved session.

Part	Content
Settings	Project root, target interpreter, goal, execution settings, tests to check, grouping method and threshold, model
Runs	Each check with its tool, scope, status (completed, timeout, cancelled, launch failure, output limit) and source (executed or imported)
Events and issues	Parsed messages; issues with their events, evidence, diagnosis and its source (rule, heuristic or model), status (open, awaiting verification, resolved, not observed, unknown) and verification (comparable, not comparable, unverifiable)
Facts and actions	Facts with their status (observed, knowledge, derived, hypothesis), rule and evidence; actions with their kind, preconditions, cost and impact
History	The plans and verdicts of every check, the environment snapshot and the verification baseline
Goal status	Unknown, blocked or achieved

The knowledge files and the tree ship inside the package. Reports are written as HTML, JSON and a graph in JSON; an exported report replaces the project path, the interpreter path and the home directory with placeholders, and redacts credentials.

6.4 The product in use

Figure 6 shows the web workspace on the supplied sample project. The first step names the removed API, the file and line, and the replacement; its details show the error, the rule and the official source behind the conclusion, and how the repair will be confirmed. Figure 7 shows the first report that a coding agent receives through MCP for one of the sealed evaluation tasks.

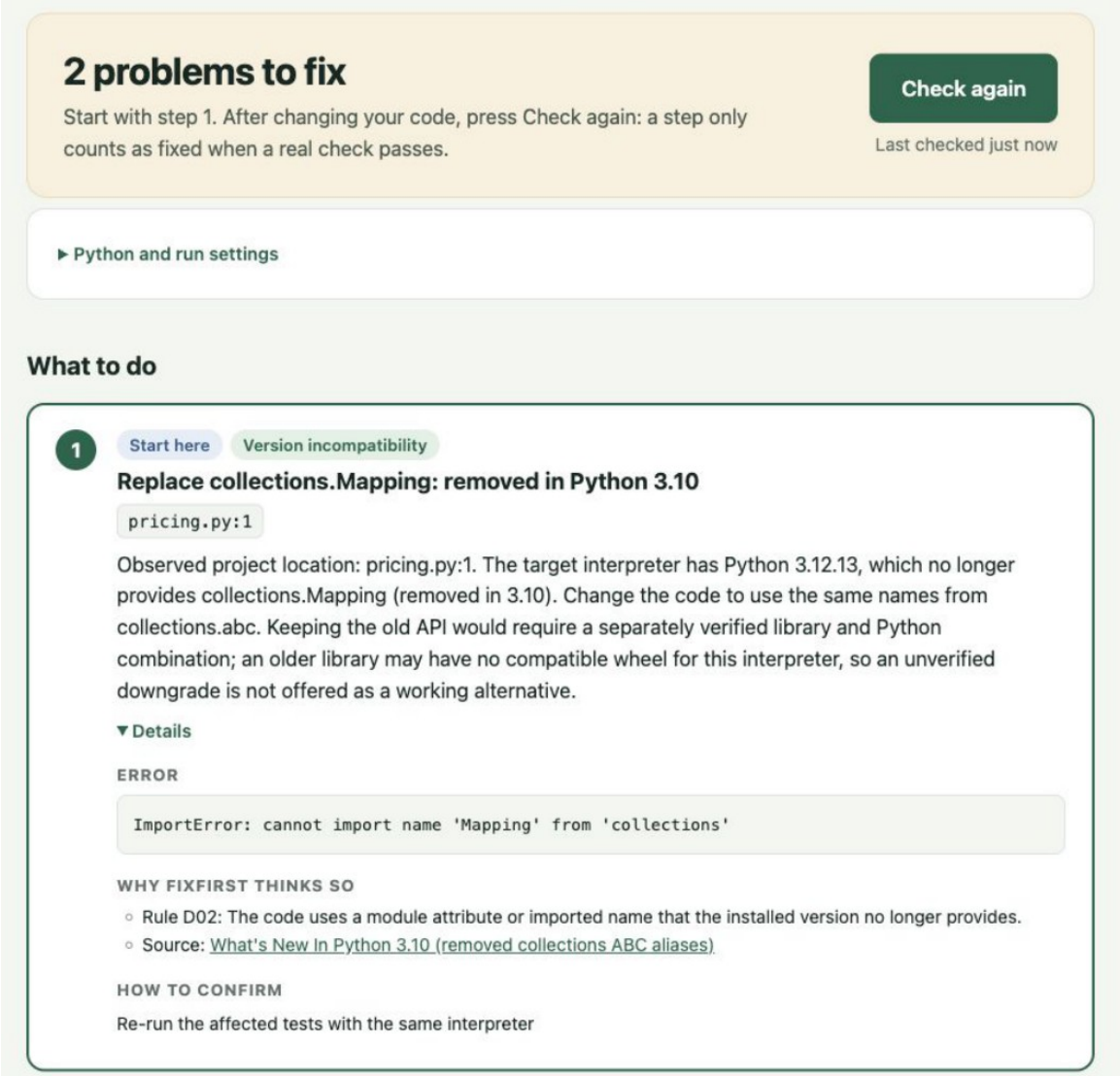


Figure 6 The web workspace on the sample project: the first step with its evidence.

```
FixFirst session session-… · project project · goal: Pass the test suite
Status: 1 problem to fix. Start with step 1. After changing your code, call check_again:
   a step only counts as fixed when a real check passes.
Python: <project>/.venv/bin/python (the project's .venv environment)

Steps, most important first:
1. Replace werkzeug.utils.bind_arguments: removed in werkzeug 2.1
   Action: Observed project location: cmdroute/dispatch.py:2. The target interpreter has
      werkzeug 3.1.9, which no longer provides werkzeug.utils.bind_arguments (removed in
      2.1). Change the code to use inspect.signature(func).bind(*args, **kwargs)
      (Signature.bind), which raises TypeError for arguments that do not fit the …
   Where: cmdroute/dispatch.py:2
   Cause: Version incompatibility (rule D02)
   Confirm: Re-run the affected tests with the same interpreter
```

Figure 7 The first report a coding agent receives through MCP on one sealed task: the action comes first, followed by the place, the cause, the rule and how to confirm it. Path shortened.

6.5 Engineering process and versions

Development was driven by concrete failures. An early evaluation on twelve real projects (Section 7.3.3) showed that recognising the right cause class is not enough, so later work added receiver attribution, constrained dependency ranges, Django settings, persistent project configuration, old test tools on new interpreters and local import roots, and the tree grew from 44 to 81 evidence features. The proposal named PyDFix's public build records as the main data source; because the tree's features come from FixFirst's own observation of a project in its environment, which historical build logs do not contain, the training data became generated cases that are actually run and whose labels are exact, and real projects and defects served for evaluation.

Each candidate version was accepted independently and frozen before it met evaluation material it had not seen (Table 15); the delivered 0.8.0 changes rc5 only where the Windows acceptance, a new pluggy release and the case generator required it.

Table 15 Frozen versions and the delivered version. The decision tree file and the feature computation are identical in all five.

Version	Tag	What changed
0.8.0rc2	`b7-freeze-20261007`	Version of the agent experiment
0.8.0rc3	`b12-freeze-20261010`	Tests to check can be named in the web form, the command line and MCP; an empty default run says so
0.8.0rc4	`b12-rc4-freeze-20261010`	The pytest plugin no longer depends on environment variables during the run
0.8.0rc5	`b12-rc5-freeze-20261010`	Older pytest supported; large suites keep their records; confirmation with the project's own pytest options
0.8.0	`v0.8.0`	Delivered version: Windows line endings, checkout bytes and session replay corrected; compatible with pluggy 1.7; rules, knowledge and tree unchanged

6.6 Safety and limits

Diagnostic checks execute project code, as the project's tests or program would, so users should check projects they trust. Source changes and installations remain visible actions of the user or agent. The optional release search uses an isolated trial environment, and its result applies only to the tested import and conditions. Inspection is bounded: when a limit is reached FixFirst gives a generic step rather than an unsupported installation command. Changes of interpreter, edits to protected tests and omitted pytest options are reported rather than silently accepted as verification. Imported logs never close an issue.

6.7 Windows

Development and all experiments used macOS. A team member installed 0.8.0rc5 on Windows 11 with CPython 3.12 from the tagged source and carried out the acceptance checklist on 13 October. All manual checks passed: installation and start, the web interface, a project in a folder whose path contains Chinese characters and a space, an installation command copied from the interface into PowerShell, export with the user name redacted, reopening a saved session, cleanup of all

processes after Ctrl+C and after closing the window, and an exact repair suggestion for a positional-only argument error. The full test suite exposed one product defect: captured output kept Windows line endings, so a program with a syntax error received no cause. Version 0.8.0 corrects that defect and several test-environment issues, and continuous integration now runs the full test suite on Windows as well as on Ubuntu. On 20 October the released 0.8.0 passed the same checks, run interactively, on a second Windows 11 machine: installation in a folder whose path contains Chinese characters and a space, the sample repair, a program with a syntax error, which now receives its cause and first step, export with the user name redacted, reopening saved sessions, and cleanup of all processes after Ctrl+C and after closing the window.

7 Evaluation and Results

7.1 Overview

The evaluations answer different questions and are reported in three layers: the machine learning evaluation of the diagnosis (Section 7.2), the system evaluation of repair and verification (Section 7.3), and the user evaluation (Section 7.4). Table 16 gives the main results; Table 17 lists all evaluations, and Appendix D the detailed tables.

Table 16 Main results.

Result	What it measures	Section
38.9% → 90.0%	Repair rate of Gemma 4 26B-A4B on 18 sealed tasks, without and with FixFirst through MCP	7.3.1
27 of 28	Unseen real code defects closed only after the real fix, with the tests named (third held-out test); none closed wrongly	7.3.2
29 of 29	Failures in the 18 unseen sealed tasks whose predicted cause class belongs to the reference repair (17 of 17 where a task has one class); first check of the frozen version	7.2.2
220 of 220	Generated cases with the right cause class, full hybrid; development data	7.2.1
−23.4%	Median repair time of six participants with FixFirst; a trend, p = 0.094	7.4

Table 17 All evaluations. “Development data” means that rules, knowledge or features were written with the material in view.

Question	Material	Version	Main result
Is the cause class right? (7.2.1)	220 generated and 30 difficult cases	rc5	Full hybrid 220 of 220 and 30 of 30; rules and knowledge alone answer 183, all correct. Development data
Is it right on unseen tasks? (7.2.2)	22 sealed tasks, first check	rc2	Full hybrid 29 of 29 failures in the 18 in-scope tasks, 17 of 17 in the single-class ones; without the knowledge graph 20; tree alone 23
Do language models do better alone? (7.2.3)	Same cases, three local models, no tools	v0.7.0 / candidate	Models 203–216 of 220 and 15–18 of 30; FixFirst 215–220 and 30
Does an agent repair more with FixFirst? (7.3.1)	22 sealed tasks, 660 local and 132 cloud runs	rc2	Gemma 4 26B-A4B 38.9% → 90.0%; Qwen3.6-35B-A3B and Qwen3.8-27B mainly faster; cloud model 100% in both arms
Is a repair recognised only when real? (7.3.2)	27 + 75 real defects (BugsInPy)	rc2–rc5	No environment command for any defect; with the tests named 9/19, 21/28, 27/28 closed correctly, none wrongly
Is it faster for people? (7.4)	6 participants, 24 tasks	rc5	Median time 23.4% lower; 5 of 6 faster; p = 0.094

Question	Material	Version	Main result
Is the first step useful on real projects? (7.3.3)	12 real projects	v0.7.0	4 correct, 1 partial, 6 generic, 1 wrong
Does following the advice repair the task? (7.3.3)	22 sealed tasks, scripted	rc2	17 of 18 in-scope tasks
Do ordering and grouping help? (7.2.5, 7.2.4)	5 multi-fault cases; 394 constructed pairs	rc5	Ordering: no difference measurable. Grouping: recall 0.986, precision 0.274

7.2 Machine learning evaluation

7.2.1 Diagnosis on generated cases

The 220 ordinary cases combine five project templates with 44 failure scenarios; 30 further difficult cases were sealed for the comparison with language models. Diagnosis is replayed from the saved observations, without running a project. The evaluation command fits a separate tree for each fold, holding out either a template or a scenario; the shipped tree is not retrained.

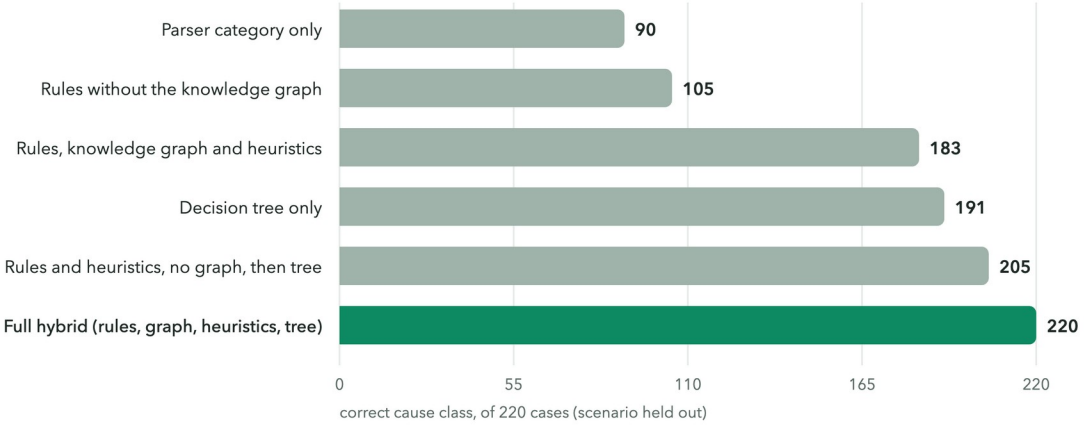


Figure 8 Correct cause class on 220 generated cases with a held-out scenario. Rules, knowledge and heuristics answer 183 cases and are right in each; the tree fills the rest. Development data.

Rules, knowledge and heuristics answer 183 of the 220 cases and are right in every one; the other 37 stay unanswered and are filled by the tree (Table 18). Without the knowledge graph the rules are right in 105, and the combination falls to 205. The tree alone depends on having seen the scenario: 220 when a template is held out, 191 when a scenario is held out. On the 30 difficult cases the hybrid is right in all, the tree alone in 15; the 25 version cases among them are decided by one heuristic written after these cases had been seen. The product itself is slightly more cautious than the evaluation command: replaying its own plan gives 215 of 220, because for five old unittest alias cases it asks for the failure to be run again instead of naming a cause.

Table 18 Correct cause class on the 220 generated cases, by variant and held-out fold.

Variant	Template held out	Scenario held out
Parser category only	90	90
Rules without the knowledge graph	105	105

Variant	Template held out	Scenario held out
Rules, knowledge graph and heuristics	183	183
Decision tree only	220	191
Rules and heuristics without the knowledge graph, then the tree	205	205
Rules, knowledge graph and heuristics, then the tree (full hybrid)	220	220

These cases are development data: rules, knowledge, heuristics and tree features were written with them in view, and a held-out fold isolates only what that fold's tree has not seen. The next section tests the same variants on tasks the product had not seen.

7.2.2 Diagnosis on unseen sealed tasks

The 22 tasks of the agent experiment (Section 7.3.1) stayed sealed until the product was frozen as 0.8.0rc2, so no rule, knowledge entry or feature was written with them in view. Each task was rebuilt from its definition, and the frozen version ran its first check with the goal of passing the tests, without any language model. Every open test failure of that check is one item. Its label is the set of cause classes of the task's reference repair, which a fixed rule assigned to each repair step, and a prediction counts as right when it is in that set. For a task with several classes this is label-set matching: it shows that each observed failure received one of the task's causes, not that every cause of the task was found. The strict reading therefore counts only the 12 tasks with a single class.

Registered before the analysis were the labels, the first checks, the six variants of Table 18, the scoring and the strict reading. The registered analysis trained its tree without the source snapshots that the shipped tree's training data contains; the shipped tree was then scored in a reading registered before it was run, with labels and checks unchanged (Appendix D.5). The intervals and the split of the answers without the knowledge graph were added after the results.

All 18 in-scope tasks produced failures to classify, 29 in all, and three of the four controls did. In the fourth control a pytest option of the project needs a plugin that is not installed; FixFirst's check, which runs pytest without the project's own options, passed and said so, and asked the user to run their usual command. That fault was not observed, so the task has no classification result.

Table 19 Cause classes on the 18 in-scope sealed tasks, first check of the frozen 0.8.0rc2, with the shipped tree. A failure is right when its predicted class is among the classes of its task's reference repair; the last column counts only the 12 tasks with a single class.

Variant	Failures right, of 29	Tasks with every failure right, of 18	Single-class tasks: failures right, of 17
Parser category only	5	4	1
Rules without the knowledge graph	10	5	2
Rules, knowledge graph and heuristics	29	18	17
Decision tree only	23	14	11

Variant	Failures right, of 29	Tasks with every failure right, of 18	Single-class tasks: failures right, of 17
Rules and heuristics without the knowledge graph, then the tree	20	12	9
Rules, knowledge graph and heuristics, then the tree (full hybrid)	29	18	17

For all 29 failures the full hybrid named a class of the task's reference repair, so every failure matched in all 18 tasks (95% interval 82–100% of tasks); in the 12 single-class tasks it named that one class for all 17 failures (95% interval 76–100% of these tasks). Rules and knowledge answer 28 of the 29; a heuristic names the cause of the remaining one, an import from a removed SQLAlchemy module. On these tasks the knowledge graph carries more of the diagnosis than on the generated cases: without it the rules answer 10, the heuristics and the tree are right in 10 of the other 19, and the system falls from 29 to 20, where it fell from 220 to 205 in development. The tree alone is right in 23 of 29 failures and in 14 of 18 tasks (95% interval 55–91%), close to its 191 of 220 with a scenario held out. In the product it decided no failure in scope, because rules and heuristics had answered every one. It decided only the two failures of the control outside the knowledge base, an Excel reader that no longer opens `.xlsx` files, and named a code defect there, which the product shows as a possible cause, not confirmed. On the two code-defect controls every variant named a code defect.

The removed APIs, fixtures and import names in these tasks were drawn in a fixed random order from the entries of the frozen knowledge base. The result therefore shows that recorded faults are recognised in projects, installed versions and messages the product had not seen; faults that the knowledge base does not record appear only among the controls.

7.2.3 Language models as baselines

Three local models, Qwen3.6-35B-A3B (6-bit)[15], Gemma 4 26B-A4B (4-bit, quantisation-aware trained)[16] and Qwen3.8-27B (4-bit)[17], answered the same cases once each, with deterministic decoding, no tools and no retries (Figure 9). Table 20 gives accuracy and macro-averaged precision, recall and F1 over the cause classes, computed from the per-class counts recorded with the results.

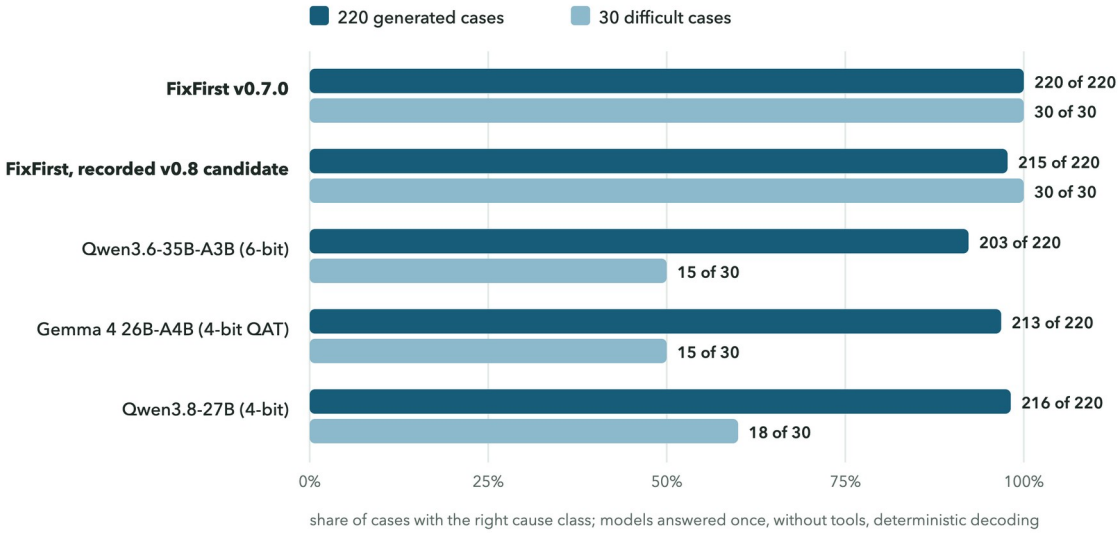


Figure 9 Right cause class on the generated cases: FixFirst against three local language models answering without tools.

Table 20 Diagnosis on the generated cases. Macro averages run over every class that occurs as a label or a prediction: the five cause classes in the 220 cases, and in the 30 difficult cases their two labelled classes and the code defects that the models predicted. Over the two labelled classes alone, the models' macro F1 on the 30 cases would be 0.786, 0.786 and 0.842.

Method	Accuracy, 220	Macro precision	Macro recall	Macro F1	Accuracy, 30	Macro F1, 30
FixFirst v0.7.0	1.000	1.000	1.000	1.000	1.000	1.000
FixFirst, recorded v0.8 candidate	0.977	1.000	0.986	0.993	1.000	1.000
Qwen3.6-35B-A3B (6-bit)	0.923	0.926	0.930	0.926	0.500	0.524
Gemma 4 26B-A4B (4-bit QAT)	0.968	0.979	0.969	0.973	0.500	0.524
Qwen3.8-27B (4-bit)	0.982	0.991	0.985	0.987	0.600	0.561

The models are strong on the ordinary cases and weak on the difficult ones: all their errors there are version incompatibilities that they did not recognise (10, 10 and 13 of 25 recognised). On twelve real projects the comparison was closer (Table 21): Qwen3.6-35B-A3B named more causes than FixFirst v0.7.0 and gave correct advice for more projects, 6 against 4. This measures category prediction and advice, not repair; Appendix D.1 gives the details.

Table 21 Language models and FixFirst v0.7.0 on twelve real projects.

Method	Correct causes of 11 faults	Correct advice of 12 projects	Missing answers
Qwen3.6-35B-A3B (6-bit)	9	6	1
Qwen3.8-27B (4-bit)	8	5	2
Gemma 4 26B-A4B (4-bit QAT)	3	0	2
FixFirst v0.7.0	8	4	0

7.2.4 Message grouping

On all 874 saved sessions the similarity step never grouped differently from exact matching. On 394 pairs constructed by rule from saved messages, TF-IDF merged 69 of the 70 pairs that belong together but also 183 of the 324 that do not: a recall of 0.986 and a precision of 0.274, below the proposal’s target of 0.90 (Table 22). In the product, a changed identifier usually moves a message into another block before any text is compared, so these wrong merges were not observed on recorded data.

Table 22 Message grouping on constructed pairs.

Change between the two messages	Pairs	Expected	Merged by exact matching	Merged by TF-IDF
One number	23	merge	0	23
One line number	10	merge	0	10
One directory name of a path	37	merge	0	36
An identifier with a near spelling	162	apart	0	144
An unrelated identifier	162	apart	0	39

7.2.5 Next-step ordering

On five generated projects with two or three faults each, FixFirst chose the right first problem in all five and never chose wrongly, but so did two naive orderings, the order of the messages and a fixed order of categories: the cases cannot separate the methods. Of FixFirst’s eleven first steps along the way, seven were an allowed repair, four named no concrete change, and none was wrong.

7.3 System evaluation

7.3.1 Agent repair with and without FixFirst

Design. The same local model works on the same task with general file and command tools, once without FixFirst and once with its MCP tools. The 22 tasks cover dependency and version bounds, removed APIs, project configuration and layout, combined faults, and two kinds of controls; the 18 tasks of the first four kinds form the scope of the main analysis. The product was frozen before the task pack was revealed. Each model ran each task five times per arm (660 runs; 90 per arm in scope), with at most 30 turns and 900 seconds. A protection rule counts any edit to tests, test collection or result settings as a failure, even if it is later undone; the same rule is stated to both arms.

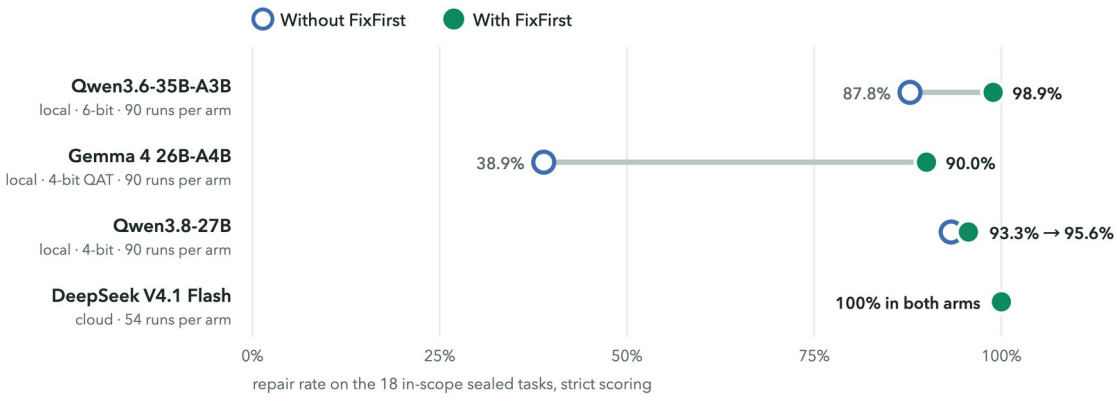


Figure 10 Repair rates without and with FixFirst. The three local models ran five times per task and arm on the frozen 0.8.0rc2; the cloud model ran separately, three times.

Table 23 Repair rates on the 18 in-scope tasks, strict scoring.

Model	Without FixFirst	With FixFirst (MCP)	Difference, 95% interval
Qwen3.6-35B-A3B (6-bit)	79 of 90 (87.8%)	89 of 90 (98.9%)	+11.1 points [1.1, 22.2]
Gemma 4 26B-A4B (4-bit QAT)	35 of 90 (38.9%)	81 of 90 (90.0%)	+51.1 points [35.6, 65.6]
Qwen3.8-27B (4-bit)	84 of 90 (93.3%)	86 of 90 (95.6%)	+2.2 points [0.0, 6.7]

Gemma 4 26B-A4B improves on sixteen tasks and declines on one, and its gain holds under every registered reading. The gain of Qwen3.6-35B-A3B depends on the strict protection rule: if a run whose only violation was another pytest option is allowed, its rates are 96.7% and 98.9%. The ten unrepaired runs of Qwen3.8-27B were all timeouts, eight of them in one task.

Efficiency is measured on tasks that both arms solved at least three times; each task contributes the ratio of its median successful runs, and Table 24 gives the geometric mean (below 1 means less).

Table 24 Turns, time and tokens with FixFirst relative to without, on tasks both arms solved.

Model	Tasks	Turns	Time	Recorded tokens
Qwen3.6-35B-A3B (6-bit)	15	0.54 [0.43, 0.69]	0.46 [0.38, 0.56]	0.36 [0.28, 0.46]
Gemma 4 26B-A4B (4-bit QAT)	7	0.55 [0.34, 0.90]	1.03 [0.72, 1.43]	0.63 [0.37, 1.04]
Qwen3.8-27B (4-bit)	17	0.82 [0.68, 0.98]	0.73 [0.60, 0.89]	0.74 [0.59, 0.91]

A cloud model. The same frozen product, runner and tasks were run with DeepSeek V4.1 Flash[18], three times per arm (132 runs). Both arms repaired 54 of 54 scope runs, so this pack measured no repair gain for a strong model. Turns, time and tokens with FixFirst were 0.82, 0.71 and 0.72 of the baseline, but unevenly: time and tokens fell on ten tasks and rose on eight. On the two code-defect controls FixFirst added overhead.

All agent results belong to the frozen 0.8.0rc2. Later candidates return the same first report for 17 of the 18 scope tasks; no agent run was repeated on them (Appendix D.2).

7.3.2 Real code defects

BugsInPy provides real defects in project code, each with the commit that fixed it and the test that shows it[12]. FixFirst checks the project before the fix and again after it. Two things are judged: that no step with an installation, upgrade or pin command is given for a code defect, and that the issue for the failing test is open before the fix and closed only after it.

Default use. On 27 defects of tqdm, tornado and cookiecutter, the frozen 0.8.0rc2 gave no environment command for any defect, but the closing check failed in all 27, mostly before diagnosis: in 24 the default pytest run never executed the failing test, because these projects run their tests in their own way. The next candidates therefore let the user name the tests, and each was tested once on defects that had been sealed after reproduction alone (Table 25).

Table 25 Held-out real defects. “Seen” means the failing test was observed; “closed” means the issue closed only after the real fix.

Test	Version	Material	rc2, default use: seen / closed	Tests named: seen / closed
1	0.8.0rc3	19 defects, 3 new projects	2 / 2	10 / 9
2	0.8.0rc4	28 defects, 6 new projects	5 / 3	27 / 21
3	0.8.0rc5	28 defects, 7 projects used before	10 / 4	**28 / 27**

With the tests named, no defect in any test received an environment command, no wrong cause was stated without the “unconfirmed” mark, and no issue was closed without the repair. The first step named a code or test defect for 62 of the 75 defects. Each test met a problem not known beforehand, which the next candidate removed: a project that replaces the environment variables during its tests, a pytest too old for an option the product passed, and a project with its own pytest options. The defects seen but not closed in test 3 are fixes that also change a file the project counts as a test; FixFirst then reports that the tests changed, which is its protection against passing by editing tests.

> The three rows are not a rising curve: versions and material differ, and the third set contains no new project. The default-use column is low partly for a reason of method, because each environment held only the packages of the defect’s own test. All defects are code defects, so these results say nothing about the other four cause classes.

7.3.3 Following the advice and first steps on real projects

Following the advice. A script followed the advice of the frozen 0.8.0rc2 on the 22 sealed tasks, without a language model, executing printed commands or preregistered readings of named source changes for at most six rounds. It repaired 17 of the 18 scope tasks; the unrepaired task received recognition of a removed SQLAlchemy module but no executable replacement. On 0.8.0rc5 the same script repairs 18 of the 22 tasks, against 17 on 0.8.0rc2; on rc5 the first step for the SQLAlchemy task gives a change that can be followed.

First steps on real projects. Before version 0.8, FixFirst v0.7.0 was evaluated on twelve real projects, one of them healthy. Its first step was rated correct in 4, partial in 1, generic in 6 and wrong in 1, while the cause class matched in 8 of 11 faulty projects; two independent raters agreed on 9 of 12 rows (Cohen's kappa 0.66). The projects were chosen from the awesome-python list by a fixed procedure; their labels were committed before the frozen v0.7.0 was run once, the second reviewer's before seeing the first's, and the scoring rubric before either reviewer scored. The gap between a right cause class and useful advice shaped the later development (Section 6.5).

7.3.4 Tests and platforms

The 1,151 test functions in 98 modules and Ruff run for pushes to main and for every pull request, on Ubuntu and on Windows with Python 3.12 (Section 6.1). The product was developed and all experiments were run on macOS; the acceptance run on Windows 11 and the defect it exposed are described in Section 6.7. When pluggy 1.7 was released during the project, these runs showed that it had moved a function of pytest's plugin machinery that the product's import observation recognises; version 0.8.0 recognises it in both places.

7.4 User evaluation

Six participants (0.5 to 4 years of Python; three had used pytest) each repaired four small failing projects with 0.8.0rc5: an import that a newer library version removed (`jinja2.Markup`), an uninstalled project under `src`, a standard library module removed in Python 3.12 (`distutils`), and a missing configuration file. Each did two tasks with FixFirst and two with only a terminal and web search, in a counterbalanced order; the protocol allowed no AI assistant in either condition. According to the experimenter, the sessions took place on 12 and 13 October with three participants working at the same time on their own computers, and no pilot session was held.

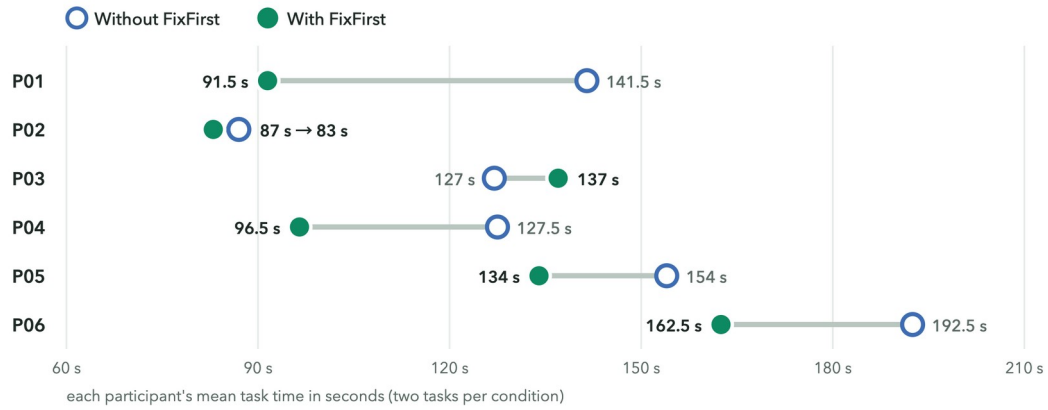


Figure 11 Each participant's mean task time without and with FixFirst. Five of six were faster with FixFirst; the difference is not statistically significant (p = 0.094).

All 24 tasks were completed. The median completion time was 104.5 s with FixFirst and 136.5 s without, 23.4% lower. Comparing each participant's mean task time, five of six were faster with FixFirst (two-sided Wilcoxon signed-rank test, p = 0.094; not significant with six participants). After the FixFirst round the participants filled in the ten-item System Usability Scale[21]; the mean score was 79.6, with individual scores from 75 to 92.5. The figures come from the experimenter's records; the cause explanations that participants wrote were not scored.

8 Discussion and Limitations

8.1 What the results mean

Specific, executable advice matters more than the right category. The first real-project evaluation matched most cause classes but gave useful first steps in only a third of the projects. Receiver attribution, constrained version ranges and persistent configuration were developed in response, and verified as product mechanisms.

On unseen tasks the knowledge carries the diagnosis. On the sealed tasks the rules and knowledge gave a cause from the reference repair for every failure in scope, and without the knowledge graph the same system was right in 20 of 29. The tree alone reached 23 of 29 and remains the fallback for failures that no rule settles.

The benefit of FixFirst depends on the agent. Gemma 4 26B-A4B gained repair capability on the sealed tasks; the two Qwen models, Qwen3.6-35B-A3B and Qwen3.8-27B, already strong, gained efficiency; a cloud model reached the ceiling without help. A strong baseline leaves little room for a tool to raise the repair rate.

On real repositories the first obstacle comes before diagnosis. A default pytest run often does not run the failing test, because projects run their tests in their own way. Letting the user or agent name the tests removed most of this in three held-out tests. A diagnostic tool must meet the project's own way of running before its rules and knowledge matter.

Not every component could be shown to help. Similarity grouping decides nothing on recorded data that exact matching does not, and the multi-fault cases cannot separate orderings. We report these as results, not as successes.

8.2 Objectives achieved and not achieved

Table 26 The objectives of Section 2.1 against the results.

Objective	Measure in the proposal	Result	Assessment
O1 Fewer repeated messages	Grouping precision 0.90 on hard cases	Precision 0.274 and recall 0.986 on 394 constructed pairs; exact matching merges none of them, and on 874 saved sessions TF-IDF never grouped differently from exact matching	Not met by the similarity method
O2 A useful first step	Better than naive orderings	5 of 5 for FixFirst and for both naive orderings: the cases cannot show a difference; 4 of 12 first steps correct on real projects with v0.7.0	Not shown
O3 Understandable, checkable advice	6–8 participants, 80% completion, median time 20% lower	6 participants, 100% completion, median time 23.4% lower ($p = 0.094$), SUS 79.6; run three at a time on their own computers, without a pilot	Met in form; the time difference is a trend

Objective	Measure in the proposal	Result	Assessment
O4 Progress only from real checks	Repairs verified on at least 8 real defects in at least 4 libraries	With the tests named, held-out defects of 2, 5 and 7 projects: 9/19, 21/28 and 27/28 closed only after the real fix, none wrongly; on 0.8.0 candidates rather than v0.7.0	Met with the tests named; default use mostly does not reach the failing test
O5 The same diagnosis for agents	An agent comparison on real projects	84 early runs on two real projects showed no measurable difference; 660 local and 132 cloud runs on 22 constructed tasks sealed until the freeze	Met on constructed tasks

The proposal also planned an ablation of the features. Grouping and ordering were ablated; the ablation without the display of evidence was not run, because it needs participants.

8.3 Comparison with the baselines

The knowledge-based core, the decision tree and the language models have different strengths (Table 27). The knowledge-based core is precise where it answers and can say what not to do, but it answers only the mechanisms it knows. The tree fills the gaps cheaply and readably, but it depends on having seen similar scenarios. The language models know more mechanisms without any upkeep, but nobody checks their answers, and on the difficult cases they recognised only half to three fifths. As coding agents they are complementary to FixFirst rather than competitors: the weakest model gained most from its evidence, and the strongest gained time.

Table 27 Strengths and weaknesses of FixFirst’s components and of the baselines, with the evidence.

Approach	Strengths	Weaknesses
Rules, knowledge graph and heuristics	Right in all 183 of the 220 cases they answer; on the unseen sealed tasks they matched the reference classes of all 29 failures; every conclusion carries its rule and source; deterministic; no environment command for any of the 102 real code defects	Answer only known mechanisms: 37 of 220 cases stay open; the knowledge needs upkeep; written with these scenarios in view
Decision tree	Fills the 37 open cases; 23 of 29 alone on the unseen sealed tasks; 13 nodes, readable in full; cheap and offline	Depends on having seen the scenario: 191 of 220 with a scenario held out; on the sealed tasks it decided only the control outside the knowledge base, wrongly; its confidence is not calibrated; used only for test failures
Language models without tools	Broad knowledge: 203–216 of 220 without any rules; no upkeep; on real projects the best gave correct advice more often than v0.7.0 (6 against 4 of 12)	Weak on difficult cases (15–18 of 30 against 30 of 30); no verification of their answers; cost and latency; answers may change between versions
Coding agents with FixFirst	The weakest model gained most (+51.1 points); the Qwen models needed 0.46 and 0.73 of the time	No repair gain for strong models; overhead on the code-defect controls

8.4 Limitations

- **Small and constructed tasks.** The 22 sealed tasks of the agent experiment and the held-out diagnosis are 18 scope tasks and four controls; the agents ran each five times locally and three times in the cloud. They are new instances of mechanisms the product covers, with removed APIs, fixtures and import names drawn from its knowledge base, not a sample of unknown real failures.
- **A strict protection rule.** It disallows some valid repairs and penalises edits that are later undone; the gain of Qwen3.6-35B-A3B depends on it.
- **Development data.** The diagnosis comparison and the difficult cases were in view while rules, knowledge and features were written.
- **Real defects are code defects only,** in environments with just the packages of the defect's own test. In default use the product still often stops early.
- **The participant study is small** and was run three participants at a time on their own computers without a pilot; every task finished in under three and a half minutes of the twelve allowed, and the time difference is not significant. It measured time, completion and usability, not whether participants understood the causes better: their written explanations were not scored.
- **Finite knowledge.** Not every replacement suggestion has an execution probe, and bounded inspection can miss unusual layouts. The ablations remove the domain knowledge as a whole, so they measure the value of its content, not the benefit of a graph over an equivalent flat lookup.
- **Platforms.** Continuous integration covers Ubuntu and Windows with Python 3.12; macOS, where all experiments ran, has no CI job, and other Python versions are not tested continuously.

8.5 Future work

Beyond the course, the most valuable work is to make default use reach the failing test on more repositories, to test previously uncovered kinds of environment failure, to run a larger participant study, including the condition without the display of evidence and a measure of how well participants understand the causes, and to add macOS to continuous integration.

9 Team Roles and Contributions

Table 28 Team roles and contributions, as recorded in the project's issues, pull requests and result files.

Member	Responsibilities	Where in this report
HE ZIHANG	• Built FixFirst: the rules, knowledge graph, decision tree, message grouping, planning and verification, and the web, command-line and MCP interfaces • Froze the candidate versions, merged the team's pull requests and released version 0.8.0 with continuous integration on Ubuntu and Windows • Ran the evaluations of diagnosis on generated cases and sealed tasks, of coding agents (660 local and 132 cloud runs), of 102 real defects from BugsInPy, and of grouping and ordering • Wrote the report and the user guide • Produced the videos: scripts, animation, editing, the live demonstrations and the agent voice-over	Chapters 4–8; Appendices C and D; both videos
WEI YI	• Reviewed the labels of the real projects independently, committing them before seeing CHI YONGJUN's • Rated the first steps of FixFirst v0.7.0 independently of CHI YONGJUN, and also rated the language models' answers • Ran the participant study: recruited the six participants, ran the 24 timed tasks and recorded the times and questionnaires • Accepted 0.8.0rc5 on Windows 11 with the acceptance checklist: a path with Chinese characters and a space, PowerShell commands, export redaction and process cleanup • Prepared the corrections for the defects this exposed, which version 0.8.0 includes, and the Windows screenshot of the user guide • Made the report template and set the structure of this report and of the two videos • Narrated the promotion video	Sections 6.7, 7.2.3, 7.3.3 and 7.4; Appendices C and D.1; the structure of this report; the promotion video
CHI YONGJUN	• Owned the independent held-out evaluation of real projects, from sampling to the published evidence • Selected the twelve projects from ten categories of awesome-python by a fixed procedure, recording all 85 candidates and every skip, and built their environments • Wrote the labels from first-hand sources, verified each in a twin by applying only the proposed first step, and recorded the AI assistance used to investigate errors • Committed the labels before the single run of the frozen v0.7.0 and the scoring rubric before scoring; scored the first steps independently of WEI YI • Fixed the held-out tool for coloured pytest output and merged the results (PR #52) • Prepared the raw sessions for the language-model baseline, checked them for label leakage and scored the model sheets • Narrated the system video	Sections 7.2.3 and 7.3.3; Appendix D.1; the AI assistance statement; the system video

References

[1] pip documentation. pip check. https://pip.pypa.io/en/stable/cli/pip_check/. Accessed 9 October 2026.

[2] S. Mukherjee, A. Almanza and C. Rubio-González. 2021. Fixing Dependency Errors for Python Build Reproducibility. ISSTA 2021, 439–451. DOI 10.1145/3460319.3464797. Author copy: https://web.cs.ucdavis.edu/~rubio/includes/issta21.pdf.

[3] Y. Peng, R. Hu, R. Wang, C. Gao, S. Li and M. R. Lyu. 2024. Less is More? An Empirical Study on Configuration Issues in Python PyPI Ecosystem. ICSE 2024. Preprint: https://arxiv.org/abs/2310.12598.

[4] Stack Overflow. 2025. 2025 Developer Survey: AI. https://survey.stackoverflow.co/2025/ai. Accessed 20 October 2026.

[5] tox-dev. pipdeptree. https://github.com/tox-dev/pipdeptree. Accessed 9 October 2026.

[6] GitHub documentation. Dependabot options reference. https://docs.github.com/en/code-security/reference/supply-chain-security/dependabot-options-reference. Accessed 9 October 2026.

[7] Renovate documentation. https://docs.renovatebot.com/. Accessed 9 October 2026.

[8] tox documentation. https://tox.wiki/en/stable/. Accessed 9 October 2026.

[9] GitHub documentation. Asking GitHub Copilot questions in your IDE. https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide. Accessed 20 October 2026.

[10] Sentry. 27 January 2026. Seer now debugs in every stage of development. Changelog. https://sentry.io/changelog/seer-now-debugs-in-every-stage-of-development/. Accessed 20 October 2026.

[11] E. Horton and C. Parnin. 2019. DockerizeMe: Automatic Inference of Environment Dependencies for Python Code Snippets. ICSE 2019, 328–338. DOI 10.1109/ICSE.2019.00047. Preprint: https://arxiv.org/abs/1905.11127.

[12] R. Widyasari et al. 2020. BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies. ESEC/FSE 2020, 1556–1560. DOI 10.1145/3368089.3417943. Author preprint: https://arxiv.org/abs/2401.15481.

[13] Model Context Protocol. Specification, revision 2025-11-25. https://modelcontextprotocol.io/specification/2025-11-25. Accessed 12 October 2026.

[14] F. Pedregosa, G. Varoquaux, A. Gramfort et al. 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html.

[15] Qwen. 2026. Qwen3.6-35B-A3B. Model card. https://huggingface.co/Qwen/Qwen3.6-35B-A3B. Run as the 6-bit MLX conversion mlx-community/Qwen3.6-35B-A3B-6bit, revision cb7e092. Accessed 12 October 2026.

[16] mlx-community. 2026. gemma-4-26B-A4B-it-qat-4bit. Model card. 4-bit MLX build of Gemma 4 26B A4B, instruction tuned, quantisation-aware trained. https://huggingface.co/mlx-community/gemma-4-26B-A4B-it-qat-4bit. Accessed 12 October 2026.

[17] Qwen. 2026. Qwen3.8-27B. Model card. https://huggingface.co/Qwen/Qwen3.8-27B. Run as the 4-bit MLX conversion lmstudio-community/Qwen3.8-27B-MLX-4bit, revision 6067b15. Accessed 12 October 2026.

[18] DeepSeek. Models & Pricing. DeepSeek API Docs. https://api-docs.deepseek.com/quick_start/pricing. Accessed 12 October 2026.

[19] pytest documentation. https://docs.pytest.org/en/stable/. Accessed 20 October 2026.

[20] Astral. Ruff documentation. https://docs.astral.sh/ruff/. Accessed 20 October 2026.

[21] J. Brooke. 1996. SUS: A 'Quick and Dirty' Usability Scale. In P. W. Jordan, B. Thomas, I. L. McClelland and B. Weerdmeester (eds.), Usability Evaluation in Industry. CRC Press, 207–212. DOI 10.1201/9781498710411-35.

[22] NUS-ISS. 2026. IRS practice module and exam briefing v2.17; IRS practice module project proposal and final presentation guidelines v016. Course materials supplied for the module.

Online sources were accessed on the dates given. The project's result files, their identities and the calculation sources are registered in `docs/report/SOURCES.md` of the repository.

AI assistance statement

Codex and Claude assisted with software implementation, documentation drafts, research checking, experiment preparation and evidence review, and with this report. Figure 1 is a concept illustration made with an image generation tool for the proposal. Local language models were evaluated as one-shot respondents and as coding agents, and a cloud coding agent was compared separately; their outputs are kept as experimental data. AI-assisted label or score drafts were reviewed by the human team, and the original independent and consensus records were kept where applicable.

For the two videos, Claude drafted the scripts with the team and produced the animated scenes and the editing; the team members recorded the narration and performed the live demonstrations; Codex operated and recorded the agent demonstration, a real run of a coding agent with FixFirst. The background music is licensed from Pixabay and credited in the videos.

The team checks generated code and claims by reading the source, executing it, running regression checks and accepting each version independently. Independent recomputation by an AI is described as such and is not presented as independent human replication. Real participants, human scoring decisions, the final Windows operation, the video narration and the personal reflections are the responsibility of the people concerned; no AI-generated narration or participant result is presented as a human contribution.

Appendix A Project proposal

The project proposal as submitted on 11 September 2026 follows on the next ten pages.

FixFirst Project Proposal

A Python Troubleshooting and Repair Verification Tool

NUS-ISS Intelligent Reasoning Systems Practice Module

11 September 2026

Practice Module Group 24

Group member	**Student ID**
HE ZIHANG	A0xxxx72R
WEI YI	A0xxxx64H
CHI YONGJUN	A0xxxx66A



Figure 1 From scattered errors to guided actions and checked results

Introduction

FixFirst will help developers handle errors in small Python projects. When package versions, tests and code checks fail together, it will bring the results into one place, recommend a useful next step and explain how to check whether that step worked.

The workflow in Figure 1 starts with a project, a Python environment and a goal. The user reviews the evidence, makes changes and reruns relevant checks. FixFirst then updates the recommendations and produces a report showing what passed and what still needs attention.

The project will combine knowledge rules, data mining and a knowledge graph. Public PyDFix records, selected BugsInPy defects and controlled examples will support development and evaluation. The intended outcome is an installable local tool, supported by reproducible experiments and a user guide.

Project goals Reduce time spent reading repeated errors, make recommendations understandable, and verify progress using actual check results.

1 Project Background and Market Context

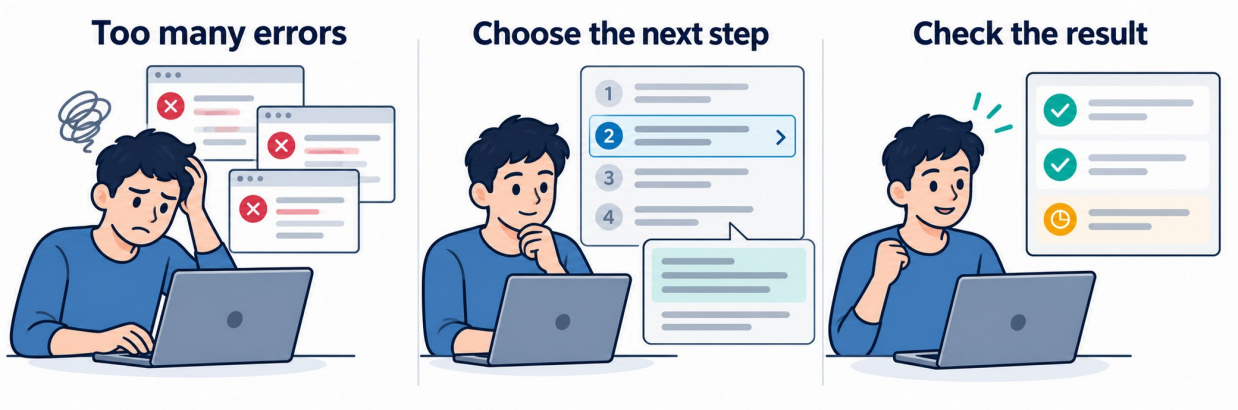


Figure 2 A student moves from a confusing error list to a clear next step

Wang takes over a teammate's Python project before an assignment deadline. He finds a package version mismatch, failing tests and several code warnings. After trying a few changes, he still cannot tell which errors are related or where to start. In Figure 2, FixFirst helps him organise the results, choose an action and check its effect.

Why this problem matters

Existing checks can identify failures, but the user still has to combine their results and decide what to do. Repeated messages may look like separate faults. A successful check may also be mistaken for proof that the whole project is working. These decisions are especially difficult for less experienced developers.

In the Stack Overflow 2025 survey, the question about frustrations with AI tools received 31,476 responses. Of those respondents, 66% cited answers that were nearly correct but still wrong, and 45% cited the extra time needed to debug AI-generated code. These findings motivate a focus on checking advice and verifying results.[1]

The PyConf study also found that package version checks alone can miss problems with the actual running environment.[2] FixFirst will therefore consider project requirements, the installed environment and test results together.

Who would use it

The initial users will be students, junior Python developers and maintainers in small teams. Likely entry points are taking over a project, moving it to a new environment and checking it before submission. Example projects and a free pilot will make the tool easy to try.

The market opportunity is a focused troubleshooting aid that fits around familiar tools. User tasks will test whether it saves effort and whether participants would use it again; the public survey alone does not establish demand for FixFirst.

2 User Needs and Product Features

Figure 3 shows how an action, its supporting evidence and a check button will appear together. Table 1 connects the main user problems to the proposed features.

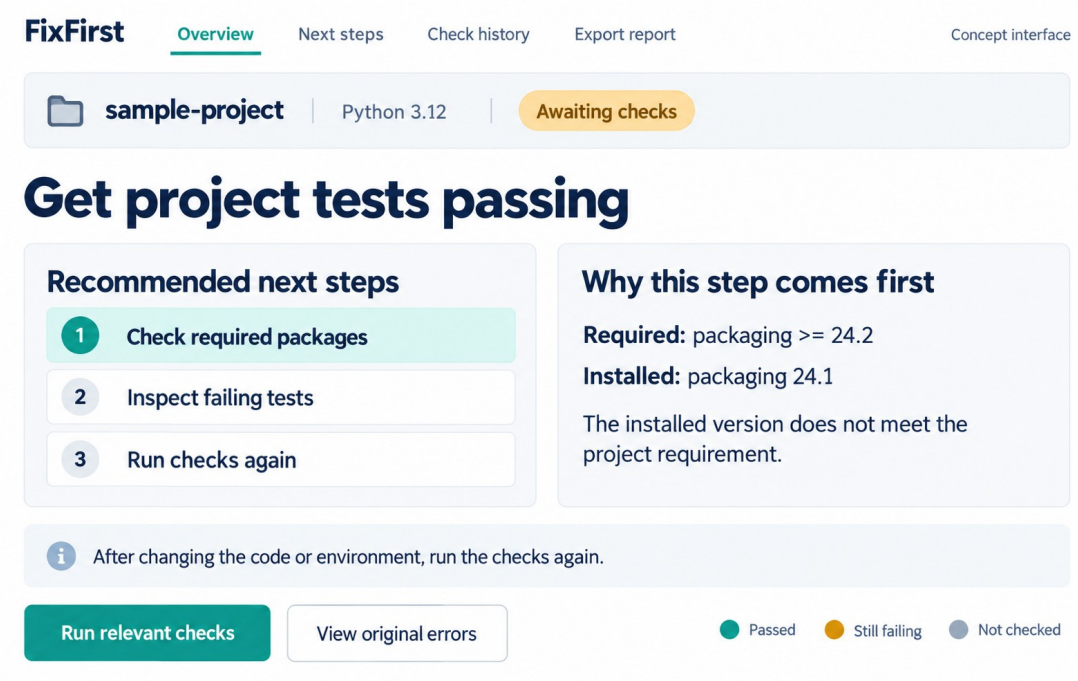


Figure 3 Concept interface with recommended actions and supporting evidence

Table 1 User pain points and the features that address them

Pain point	Feature	User benefit
Results are scattered.	Collect environment, dependency, test and code-check results.	Less switching between tools.
The same error appears repeatedly.	Group similar messages and retain the originals.	Fewer messages to read.
The next step is unclear.	Order actions by the selected goal and required conditions.	A clearer starting point.
Advice is hard to trust.	Show requirements, environment facts and source errors.	Advice that can be checked.
The effect of a change is uncertain.	Rerun checks and update only the covered issues.	Clear passed and unchecked states.
The process is hard to explain.	Export the environment, check history and remaining issues.	Easier help and handover.

A local browser interface and a command-line entry point will share the same workflow. Users will make code or environment changes themselves, then run checks and save the updated report.

3 Market Research and Competitive Analysis

FixFirst will build on established checking tools. Table 2 compares their documented roles with the product's intended focus; the comparison is a positioning analysis, not a performance ranking.

Table 2 Related tools and the proposed focus of FixFirst

Existing solution	Main capability	FixFirst focus
pip[3] uv[4]	Check installed dependencies and manage package versions and project environments.	Combine requirements and runtime results to suggest follow-up checks.
pytest[5]	Run tests, explain failures and rerun selected tests.	Connect test failures with environment evidence and show what a rerun covered.
Ruff[6]	Check code, format files and fix supported issues.	Include code warnings in a wider troubleshooting flow, ordered by the user's goal.
GitHub Copilot[7]	Agent mode can inspect a task, edit files and run commands through multiple steps.	Provide a local core workflow with visible evidence and check coverage, without requiring a cloud model.
Sentry Seer[8]	Use code and runtime information to investigate and fix issues across development stages.	Offer a simple entry point and shareable records for small Python projects.
PyDFix[9]	Detect dependency errors and iteratively patch versions to make Python builds reproducible.	Guide users across mixed checks, with goal-based actions, user-controlled changes and explicit verification.

Choose actions that help the current goal

A user may want tests to load, code checks to pass or the project test suite to pass. The same errors may need a different order for each goal. For example, a missing prerequisite should be investigated before repeatedly running tests that cannot start.

Explain the recommendation and verify its effect

Each recommendation will point to a project requirement, an environment fact or a check result. After a change, the next check will determine progress. Passing one check will confirm only its own scope, while the report will retain unresolved and unchecked items.

The project's contribution is this focused combination of guidance, evidence and verification. Comparisons will test whether it improves the task rather than assuming that the combination is effective.

4 Project Scope and System Design

Figure 4 shows the flow from checks to advice, with new results updating the next step.

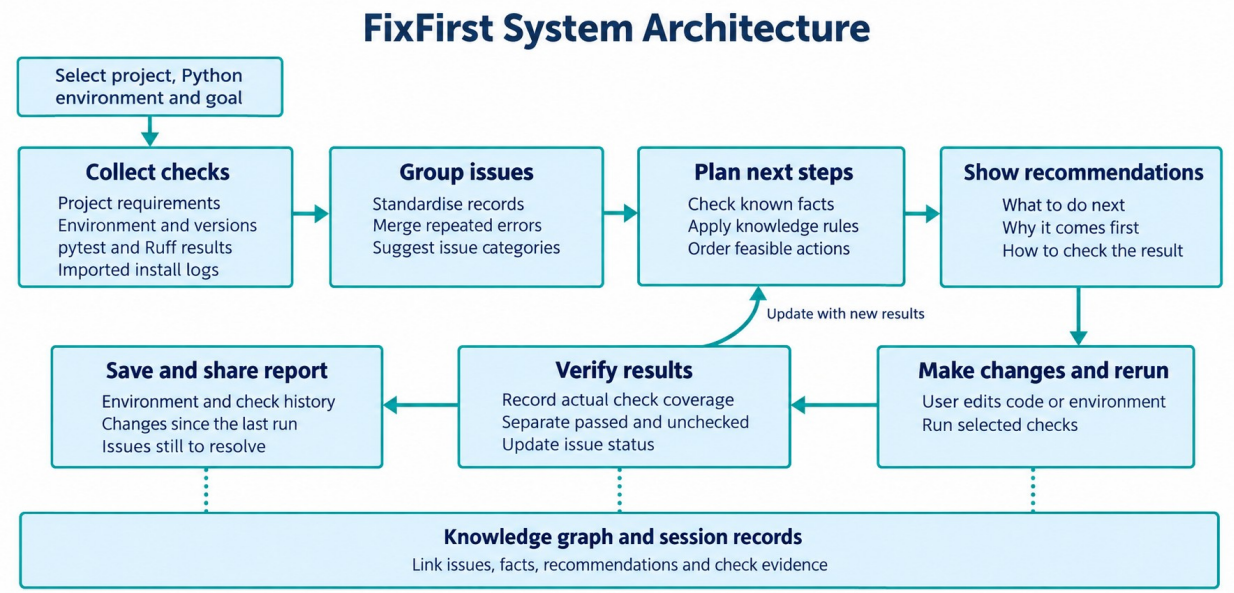


Figure 4 System architecture and the feedback from verification to planning

Turn check results into useful actions

The system will read package requirements, inspect the selected Python environment and collect dependency, pytest and Ruff results. Imported installation logs will provide further evidence. A common format will retain original messages and support grouping.

Rules will combine the evidence and user goal to recommend actions, each with a reason and a way to check the outcome. Users will make the changes and run selected checks.

Use new results to update the plan

Verification will record what ran and what passed, then update issue status and the next step. Users can continue towards the selected goal or save the report to resume later.

Initial scope

The first release will cover small Python projects on macOS and Linux, including third-party dependencies, test loading, test execution and code checks. Shared Python modules will serve the interface, command line and experiments.

Records will stay local by default. Users will see the check scope and can stop execution. Unsupported errors will retain their original messages and be flagged for further investigation.

5 Intelligent Reasoning Techniques

The project will apply three of the course's technique groups.[10] Table 3 links each group to a concrete method and a user-facing function.

Table 3 Course techniques and their role in the product

Course technique group	Methods	Product role
Decision automation and knowledge-based reasoning	Forward chaining; action rules with prerequisites; goal-based heuristic ordering.	Derive feasible next actions from known facts and rank them for the selected goal.
Knowledge discovery and data mining	TF-IDF and cosine similarity; grouping with structural constraints; Gini decision tree.	Reduce repeated messages and suggest likely issue categories.
Cognitive techniques and knowledge representation	Typed knowledge graph; breadth-first traversal; common questions mapped to graph queries.	Link issues, recommendations and evidence to explain what to do and what remains unchecked.

Knowledge rules guide the next step

Forward chaining starts from confirmed facts and applies rules to derive an action. If the installed version does not meet a project requirement, the system can recommend checking that mismatch. Actions must meet their prerequisites; feasible actions will then be ordered by their likely contribution to the goal and estimated effort.

This approach suits a small, explainable set of troubleshooting actions. It also allows the plan to change after every check. The system will recommend a practical next step rather than claim a globally optimal repair sequence.

Data mining reduces repeated information

TF-IDF converts error text into comparable features, and cosine similarity measures how similar two messages are. Checks on the stage and error type will help prevent unrelated failures from being merged. A Gini decision tree will separately use structured error features to suggest a category. Its results will be compared with a rule-based classifier.

A knowledge graph explains the evidence

The graph will connect issues, known facts, recommendations and check records. Breadth-first traversal follows these links to answer supported questions such as why an action comes first or what has not been checked. Confirmed facts and candidate explanations will be distinguished; similar wording alone will not be treated as proof of causation.

The evaluation will measure each module and the complete workflow, so the role of each technique can be assessed.

6 Data Sources and Collection

The main data source will be existing public PyDFix logs and experimental records. BugsInPy will support reproducible checks on real defects, while a limited set of controlled faults will fill coverage gaps. Table 4 explains what each source contributes.

Table 4 Available data and its intended use

Data source	Available material	Use in FixFirst
PyDFix public research artefact[9]	Build logs, dependency-error analysis, candidate packages and repair outcomes. Two published analysis tables contain 506 and 1,421 data rows.	Primary material for parsing logs, studying repeated errors and preparing reviewed dependency-error examples.
BugsInPy benchmark[11]	The original benchmark reports 493 real bugs from 17 Python projects, with tests and tools for reproduction.	Select compatible defects and fixed versions to test recommendations and verification on real projects.
Controlled supplementary cases	Small Python projects with deliberately introduced single or combined faults and a known working version.	Cover gaps such as code warnings, missing environment settings and partial reruns, with known expected outcomes.

What the public data makes possible

PyDFix provides a sizeable starting pool: the two analysis tables contain 1,927 records in total. These are candidate experimental records, not 1,927 independent, manually labelled Python bugs. Related runs and versions may overlap, and the selected material must be checked for language, relevance and duplication.[9]

Saved logs can support parsing and message analysis without rebuilding every historical environment. Live verification will use a smaller subset that can be reproduced reliably. This keeps the data work practical while preserving a separate test of whether a repair really changes the result.

Planned coverage and acceptance conditions

The curated collection will target at least 60 reviewed cases across five areas: environment setup, dependencies, test loading, test execution and code checks. Public cases will be used first, with controlled cases added where necessary. This is an initial evaluation collection, not a claim of broad production coverage.

For live verification, the target is at least eight reproducible real defects across four third-party libraries. A case will be admitted only when the same relevant test fails on the defective version and passes on the fixed version. Historical environments that cannot be reproduced will be reported separately.

7 Data Preparation and Model Training

Public records provide examples of failures, but they do not directly provide every label needed by FixFirst. The project will therefore prepare a reviewed dataset before training the classifier and evaluating recommendations. The data sources in Table 4 will be processed through four steps.

1 Collect and standardise records

For each selected case, the dataset will record its source, upstream project or defect, environment, triggering command and original output. Logs will be converted into a common format while retaining the raw evidence. Download scripts and source versions will make the collection traceable. Reuse conditions will be recorded with the materials.

2 Add and review the labels

The team will label issue categories, messages that belong together, reasonable next actions and the checks needed to confirm progress. Another member will review the labels. PyDFix's suggested packages and reported repair results will be treated as candidate annotations; they will not automatically become the correct answers for our evaluation.

Where several actions are reasonable, the labels will accept more than one valid next step. A historical fix can help establish the cause, but it will not be supplied as an input to the system being evaluated.

3 Train the classifier and prepare the rules

The Gini decision tree will learn issue categories from labelled, structured features such as the check stage, error type and package evidence. TF-IDF will be fitted on training text, and grouping thresholds will be adjusted on separate validation cases. Knowledge rules will be written from documented conditions and reviewed examples.

The project will not train a large language model. The learning component has a narrow task: helping classify errors. Rules and the knowledge graph will organise actions and explanations. When a category is uncertain, the system will retain the original evidence and request a further check.

4 Evaluate on cases kept out of training

Training, validation and test cases will be separated by upstream project or defect group. Versions and message variants of the same fault will stay together. Because PyDFix includes BugsInPy-derived material, overlapping cases will be grouped across sources before splitting the data. Test cases will not be used to tune rules or thresholds.

The final dataset description will report independent defects, repeated runs and controlled variants separately, together with rejected and unreproducible cases. Real and controlled results will also be reported separately so that convenient examples do not hide gaps in real-world coverage.

8 Evaluation Plan and Expected Deliverables

Evaluation will test both technical correctness and usefulness to users. Table 5 sets out the comparisons. All numerical values below are proposed targets; the final report will show measured outcomes, sample sizes and failures.

Table 5 Evaluation questions and planned comparisons

Evaluation area	Comparison	Main measures
Message grouping	Compare with grouping only identical text.	Precision, recall and F1; target grouping precision of at least 0.90.
Issue classification	Compare the Gini decision tree with rules on the same held-out cases.	Per-category results, Macro-F1 and common misclassifications.
Next-step guidance	Compare with message order and a fixed order of issue categories.	Whether the first action is reasonable; invalid attempts before the goal is reached.
Verification	Check full runs, partial reruns and unchecked items.	Correct status updates; no unsupported completion claim in the specified boundary tests.
User troubleshooting	Compare ordinary tools with FixFirst on matched tasks.	Completion rate, time, unsuccessful attempts and understanding of the advice.

User study and outcome targets

Six to eight participants with basic Python experience will complete two tasks of similar difficulty but different content. Tool order will be counterbalanced. The targets are an 80% task completion rate and a 20% reduction in median time for successfully completed tasks, compared with ordinary tools. Failures and timeouts will be reported alongside timing results.

Ablation experiments will remove message grouping, goal-based ordering or evidence display one at a time. These comparisons will help identify which functions contribute to the result. The small user study will indicate usability trends rather than establish an effect for all developers.

A usable product and reproducible evidence

The final delivery will include the installable local tool, example projects, a user guide, data with source notes, reproduction and evaluation scripts, and a project report with demonstration materials. Users should be able to select a project, review advice, make a change, rerun checks and export a useful report.

This delivery will demonstrate business value through user tasks, system design through the reasoning workflow, and implementation quality through an end-to-end product. The experiments will show how the three course technique groups contribute to the outcome.[10]

References

Online sources were accessed on 11 September 2026. Source titles link to the original materials. Superscript citations and figure and table references link within this document.

[1] Stack Overflow (2025). Developer Survey 2025 AI.

[2] Peng, Y., et al. (2024). Less is More? An Empirical Study on Configuration Issues in Python PyPI Ecosystem. ICSE 2024.

[3] Python Packaging Authority. pip check documentation.

[4] Astral. uv documentation Locking and syncing.

[5] pytest developers. pytest documentation.

[6] Astral. Ruff documentation.

[7] GitHub. Asking GitHub Copilot questions in your IDE. Includes agent mode.

[8] Sentry (27 January 2026). Seer now debugs in every stage of development.

[9] UC Davis PLSE. PyDFix research artefact and documentation. Source revision b276d79e7aff. Counts exclude CSV headers and refer to records, not unique defects. BugsInPy analysis table. BugSwarm analysis table.

[10] NUS-ISS (2026). IRS practice module and exam briefing v2.17; IRS practice module project proposal and final presentation guidelines v016. Course materials supplied for the module.

[11] Widyasari, R., et al. (2020). BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies. ESEC/FSE 2020. Official benchmark repository.

AI assistance statement

OpenAI Codex was used to assist with source research, translation, drafting and document layout. The built-in image generation tool produced the English concept illustrations. The team is responsible for the final content, references and quality. AI assistance used in development and experiments will be recorded separately.

Appendix B Mapping to the MR, RS and CGS courses

The briefing asks for the system's functions to be mapped to the knowledge, techniques and skills of the modular courses[22].

Table 29 System functions mapped to the courses.

System function	Knowledge, technique or skill	Course
Evidence to causes and actions	Production rules, forward chaining, explanation with provenance	Machine Reasoning
Learned possible cause	Supervised Gini decision tree; held-out validation and ablation	Machine Reasoning; Reasoning Systems
Action priority and release search	Decisions under goals and constraints; bounded search with a trial budget	Reasoning Systems
Domain and session relationships	Knowledge graphs and graph traversal	Cognitive Systems
Repeated error grouping	Text features, TF-IDF similarity, complete-link clustering	Cognitive Systems
Web and MCP interaction	Explanations, feedback after each check, an interface for software agents	Cognitive Systems

The bounded version search is not presented as A* or evolutionary computing.

Appendix C Installation and user guide

This guide covers FixFirst 0.8.0, tag `v0.8.0`. Use the browser or command line to diagnose your project, or connect the same checks to a coding agent through MCP. You make the changes; FixFirst checks their effect.

C.1 What you need

Use macOS, Linux or Windows 10/11 with Python 3.10 or later to install FixFirst. Python 3.12 is a convenient choice. Git is optional if you download the source ZIP instead. First installation needs internet access.

There are **two Python environments**: FixFirst's `.venv` runs the tool; your project's environment runs the code you want to check. Select the project's Python in the interface. The target can be a supported Python 3.9 to 3.14 environment, including venv or Conda. Do not install project dependencies into FixFirst's environment by mistake.

Checks run project code, just as its tests or program do. Use projects you trust. FixFirst's normal checks do not need a language model or API key.

C.2 Install and start

If you received the repository as a ZIP, extract it and open a terminal in its `SystemCode` folder, which holds the code in the course layout. Skip the `git clone` and `cd FixFirst` commands below.

C.2.1 macOS and Linux

Open a terminal and run:

```
git clone --branch v0.8.0 --depth 1 \
  https://github.com/ZihangHe13123/FixFirst.git
cd FixFirst
bash scripts/setup.sh python3.12
.venv/bin/python -c \
  "import importlib.metadata as m; print(m.version('fixfirst-local'))"
.venv/bin/fixfirst serve
```

The version check prints `0.8.0`. If your Python has another command or location, replace `python3.12` with it. The setup script creates the tool's environment and installs its dependencies. Keep the terminal open while using the browser. Press Ctrl+C in that terminal to stop the server.

C.2.2 Windows PowerShell

Open PowerShell and run:

```
git clone --branch v0.8.0 --depth 1 `
  https://github.com/ZihangHe13123/FixFirst.git
cd FixFirst
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
.\.venv\Scripts\python.exe -c `
  "import importlib.metadata as m; print(m.version('fixfirst-local'))"
.\.venv\Scripts\fixfirst.exe serve
```

The execution policy option applies to that setup process. It does not permanently change your machine's policy. After setup, `start-fixfirst.bat` is another way to start the application. On macOS, `start-fixfirst.command` is available.

The terminal prints the local browser address. Only this computer can reach the normal server. Open the printed address if the browser does not open automatically.

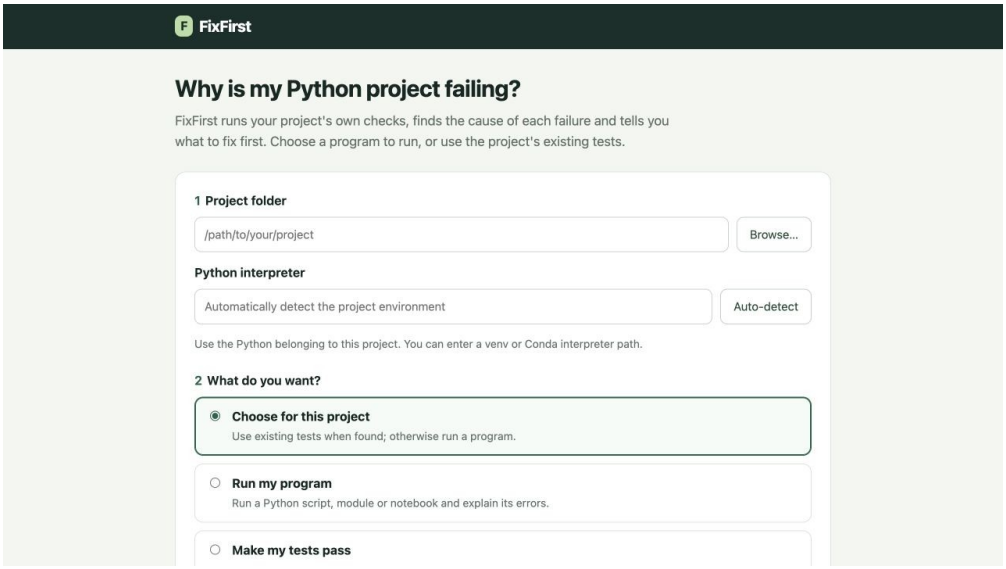


Figure 12 Choose a folder, the project's Python and the check you want to perform.

C.3 Try the supplied sample

On the start page choose **Open a sample project**. FixFirst creates a separate example with four faults and a `FIXES.md` file. Do not edit the original `examples/playground` template.

The first check finds two imports that stop test collection:

1. In `pricing.py`, change `from collections import Mapping` to `from collections.abc import Mapping`.
2. In `reports.py`, change `from helper import double` to `from helpers import double`.
3. Press **Check again**. The imports are verified, and two runtime failures become visible.
4. In `stats.py`, replace `numpy.float(sum(values))` with `float(sum(values))`.
5. For this sample only, replace `os.environ["SHOP_API_TOKEN"]` in `orders.py` with `os.environ.get("SHOP_API_TOKEN", "demo-token")`. This is a dummy value for the example, not a real credential.
6. Press **Check again**. The goal should now show **All tests pass**, with the earlier issues under **Fixed and verified**.

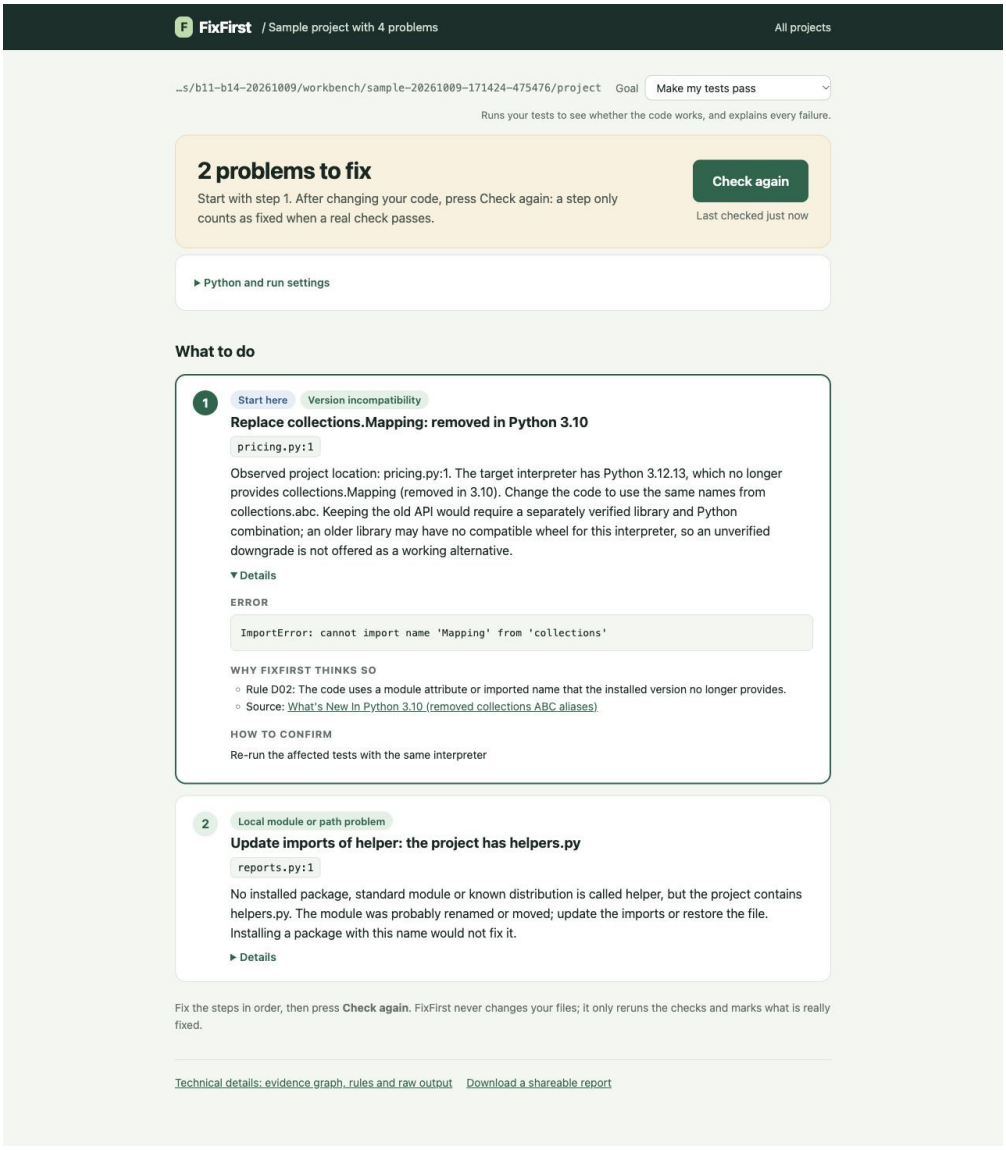


Figure 13 Details shows the observed error, supporting rule, documentation and confirmation check. The text instructions above give the edits without requiring you to read the screenshot.

FixFirst does not repair those files when you press a button. You edit them, and the next check verifies what happened. A later check may reveal a failure that an earlier collection error had hidden.

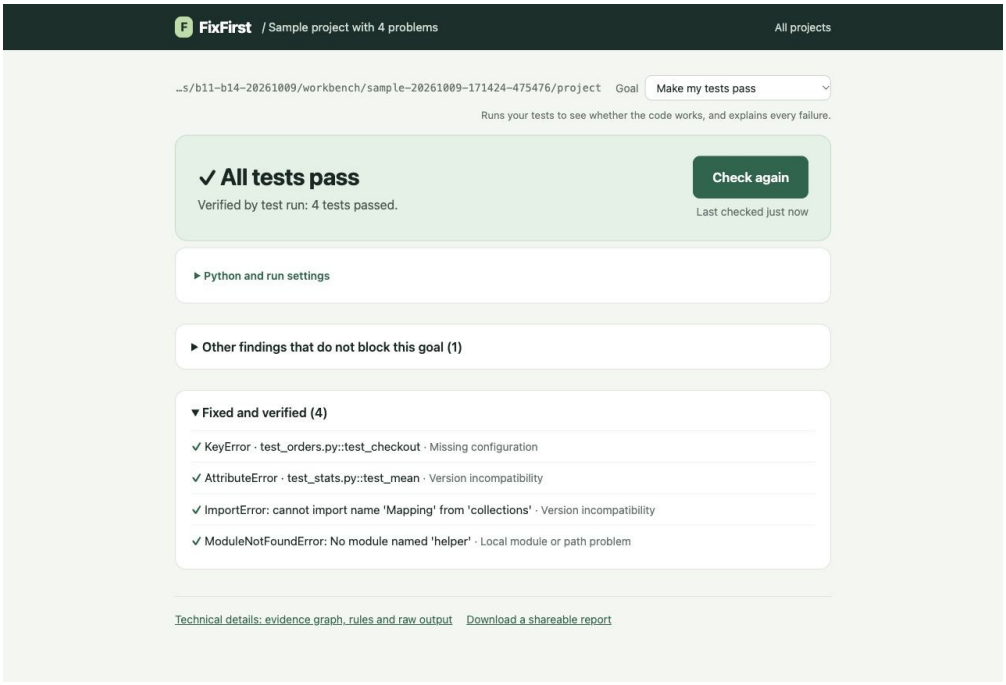


Figure 14 A completed passing check verifies the sample repair.

C.4 Check your own project

Enter your **project folder**, not a single source file. Select its Python executable, for example `.venv/bin/python` on macOS/Linux or `.venv\Scripts\python.exe` on Windows. **Auto-detect** can find common environments; check that the selected path is the one you normally use.

Choose for this project suggests a check from static file discovery. You can override it:

- **Make my tests pass** runs a pytest project.
- **Run unittest tests** uses standard-library unittest; pytest is not required.
- **Run my program** runs a script, module or optional notebook.
- **Just get the tests to load** checks collection without executing test bodies.
- **Clean up code-check warnings** uses Ruff for the code-check goal.

Name the tests when the project has its own way of running them. By default **Make my tests pass** runs pytest from the project folder with no selection. Many repositories do not work that way: their test files have other names, some need packages you have not installed, or a helper file stops collection. FixFirst then reports **Default pytest did not find any tests** or a failure that has nothing to do with yours. In that case fill in **Tests to run (optional, one file or pytest node ID per line)**, for example `tests/test_orders.py` or `tests/test_orders.py::test_total`. The selection is saved with the session and used again by **Check again**. Its result covers the named tests only.

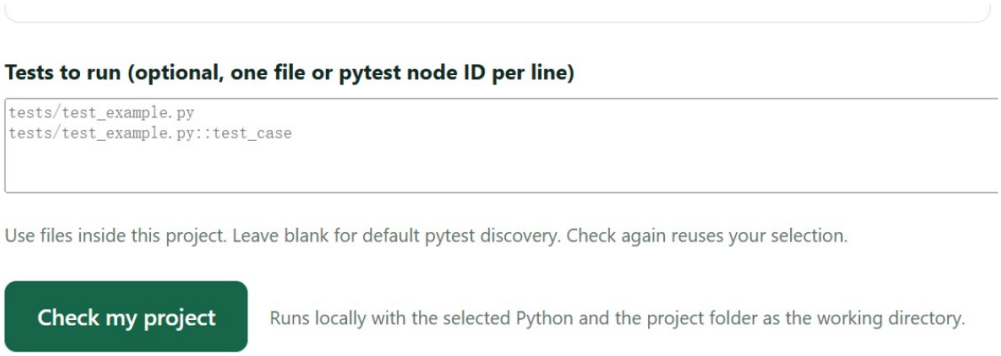


Figure 15 The optional *Tests to run* field and the *Check my project* button, here in 0.8.0rc5 on Windows 11.

For a program, select the entry you normally run. Put each argument on a separate line and supply standard input when the program uses `input()`. A complete exit code 0 produces **Program completed successfully**. This verifies that entry and input, not all business logic or assignment answers.

Follow the first concrete step, then press **Check again**. Installation commands use the selected project interpreter. Review the command and the project's declarations before executing it. A conflicting version pin can require a source migration or declaration review instead of an installation command.

For an import path or Django setting, the product may propose a persistent project configuration. Use the file it names and preserve unrelated existing options. Some packaged projects also need installation to provide their own distribution metadata; setting PYTHONPATH alone is not always sufficient.

Notebook support is optional. In the FixFirst checkout, install the driver into the tool environment with:

```
.venv/bin/python -m pip install -e '.[notebooks]'
```

The selected notebook environment also needs `ipykernel`. Cells execute in a fresh kernel. Interactive input, GUI applications and indefinitely running services need another workflow.

C.5 Understand the result

What to do lists actions that affect the selected goal. When optional actions are present, the main list is labelled **Must fix**, with a separate **Optional** section. Optional items do not establish that the program is broken. A likely cause from a heuristic or tree is not a rule-confirmed diagnosis. Read **Details** to see the evidence, rule and source.

Technical details contains the session graph and raw check output. **Find it**, when offered, tries a bounded set of older releases in an isolated environment and needs internet access. A successful trial verifies its particular import; it does not prove every test will pass with that release.

Changing the entry, input or interpreter requires fresh verification. A partial, timed-out or cancelled check does not close an original issue. When the project sets its own pytest options, FixFirst checks with fixed options first and then confirms a repair by a second run that uses the project's

options. The issue is closed only if that run executes the same tests and all pass (**Verified with the project's pytest options**); otherwise you see **Project pytest options were not verified** with the reason, and should run your usual test command yourself.

Three messages mean that the check itself was incomplete, not that a cause was found. **Default pytest did not find any tests**: name the tests as in Section C.4. **Test records are incomplete**: the suite produced more records than FixFirst keeps; the failures seen are retained, and a smaller test range gives a complete check. **pytest stopped with an internal error**: pytest itself failed before finishing; the cause is not confirmed.

Editing tests, conftest files or protected selection settings is not taken as repairing the original failure. If you intentionally change the project specification, the human interface can accept a new baseline. An MCP agent cannot use that operation.

If a step only asks you to inspect the project's code or tests, FixFirst does not yet have a supported environment repair for that failure. It is not a general business logic repair system.

C.6 Use the command line

The following commands run from the FixFirst checkout. Replace `/path/to/project`, the target Python path and `SESSION_ID` with your values. On Windows, use `.\.venv\Scripts\fixfirst.exe` and Windows paths; quote paths containing spaces.

```
.venv/bin/fixfirst init /path/to/project \
  --python /path/to/project/.venv/bin/python --goal pass_tests
.venv/bin/fixfirst scan SESSION_ID
.venv/bin/fixfirst show SESSION_ID
.venv/bin/fixfirst report SESSION_ID --open
.venv/bin/fixfirst export SESSION_ID --output shared.html
```

To name the tests on the command line, add `--tests` with files or node IDs:

```
.venv/bin/fixfirst init /path/to/project \
  --python /path/to/project/.venv/bin/python --goal pass_tests \
  --tests tests/test_orders.py::test_total
```

For a project without tests:

```
.venv/bin/fixfirst init /path/to/project \
  --python /path/to/project/.venv/bin/python \
  --goal run_project --script main.py
.venv/bin/fixfirst scan SESSION_ID
```

`--module package.main` selects a module entry. Repeat `--arg VALUE` for arguments; use `--arg=--flag` for an argument beginning with a dash. `--stdin-file answers.txt` supplies saved input. `--unittest-dir tests` selects unittest discovery.

The default CLI store is `.fixfirst/` in the working directory. Put `--store /path/to/store` **before** the subcommand to select another location. A default scan refreshes declarations and environment evidence as well as the goal's check. Use the product's normal **Check again** route after changes.

C.7 Connect a coding agent through MCP

Use a client that supports an MCP stdio server. For clients accepting an `mcpServers` JSON object, configure the **absolute** path to FixFirst's Python:

```
{
  "mcpServers": {
    "fixfirst": {
      "command": "/absolute/path/to/FixFirst/.venv/bin/python",
      "args": ["-m", "fixfirst", "mcp"]
    }
  }
}
```

On Windows, the command can be `C:\\dev\\FixFirst\\.venv\\Scripts\\python.exe`. Escape backslashes in JSON. Your client's configuration location and schema can differ; adapt the server command to its MCP settings. FixFirst itself needs no model provider credential.

Ask the agent to diagnose the project, act on the first supported step and check again. The tools are:

Tool	Use
`diagnose`	Run checks for a project and goal; return ordered steps and a session identifier
`check_again`	Repeat the saved checks after the agent changes files
`explain`	Read the evidence and rationale for a step without running checks

Example `diagnose` arguments for tests:

```
{
  "project": "/absolute/path/to/project",
  "python": "/absolute/path/to/project/.venv/bin/python",
  "goal": "pass_tests"
}
```

To limit the check to particular tests, add `"tests": ["tests/test_orders.py::test_total"]`. The selection is kept for `check_again`.

For a script, use `goal: "run_project"` and add `execution: {"kind":"script","entry":"main.py"}`. Omit `execution` for `pass_tests`, `collect_tests` and `check_style`. If the tool says to remove an extra parameter, correct it and retry the same goal.

The agent performs source edits and dependency installations; MCP does not silently do those repairs. Use the returned `session_id` with `check_again`. Stop after a verified goal, or when the tool identifies a boundary that needs human investigation. Do not keep calling it as an algorithm repair engine.

MCP sessions normally live in `~/.fixfirst/sessions`. To inspect them in the browser, start the web server with that store:

```
.venv/bin/fixfirst --store ~/.fixfirst/sessions serve
```

C.8 Troubleshooting

Python is not found. Install a supported Python and pass its executable to the setup script. For checks, select the project's interpreter rather than the tool's interpreter.

PowerShell blocks setup. Run the process-scoped `-ExecutionPolicy Bypass` command in Section C.2. Do not permanently relax machine policy just to install this tool.

The browser does not open or a port is busy. Use the address printed in the terminal. To choose another port, run `fixfirst serve --port 8765`; `--no-open` starts without opening a browser.

Checks are slow. Test execution, package metadata, antivirus scans and optional release trials can take time. Keep the selected environment local where possible. A timeout is an incomplete check, not a successful repair.

The project has no pytest. A program or unittest goal does not require it. A pytest project needs pytest in its own environment, not merely in FixFirst's environment.

Find it cannot access the index. Check network access and the project's supported distribution. The normal local diagnosis remains available. A package with only a source distribution may need a compiler or an alternative installation workflow.

Paths contain spaces or Chinese characters. Enter the real folder and Python path, and quote command-line paths. Windows output commands are written for PowerShell.

Share a session. Use **Download a shareable report** or `fixfirst export SESSION_ID --output shared.html`. Inspect the exported copy before sharing it. Keep the local store for follow-up checks.

C.9 Uninstall and retain your work

Stop the running server and remove FixFirst's installation folder when you no longer need it. Your checked project's source is separate and should not be deleted. CLI/web session records may be in `.fixfirst/` or your custom store; MCP sessions may be in `~/.fixfirst/sessions`. Back up or export records you want to keep before removing those stores.

The screenshots of the sample workflow were taken with 0.8.0rc2 and the one of the tests field with 0.8.0rc5 on Windows 11; the sample repair they show is the same in 0.8.0.

Appendix D Experiment details

D.1 Language models without tools on real projects

Each local model answered twelve prompts on the real projects of Table 21 without tools or a chance to run its advice. Five of the 36 requests produced no usable answer for technical reasons and stay in the denominator.

Three of FixFirst's eight matching causes were marked possible or uncertain. Macro-F1 over the eleven faulty projects is 0.827, 0.798, 0.317 and 0.813 in the order of the table; counting the healthy project, Qwen3.6-35B-A3B drops to 0.800. Two raters agreed on 29 of 31 advice ratings (Cohen's kappa 0.904); the first rater's rows were posted publicly before the second rater returned their sheets, so independence is declared but cannot be shown from the records. The models received a bounded, redacted presentation of the initial checks, while the product could also use its release search. The three local models are Qwen3.6-35B-A3B in a 6-bit MLX conversion[15], Qwen3.8-27B in a 4-bit MLX conversion[17] and Gemma 4 26B-A4B, instruction tuned and quantisation-aware trained, in a 4-bit MLX conversion[16]. On the generated cases the models answered 750 requests, 250 per model (Section 7.2.3).

D.2 Agent experiment

Conditions. Both arms share the task descriptions, the budget (30 turns, 900 seconds, 32,768 output tokens, up to three reminders) and the protection rules. The protection rule allows only `pythonpath` and `DJANGO_SETTINGS_MODULE` changes to the pytest configuration. Paired arms share a seed identifier and alternate their order; prompt caching starts fresh for each run; sampler settings differ by model but are the same for both arms of a model.

Sensitivity of Qwen3.6-35B-A3B. Allowing a run whose only violation was another pytest option gives 96.7% and 98.9% (+2.2 points, interval −2.2 to 6.7); the final compliant state reading gives 94.4% and 100.0%. Both readings were registered and are reported beside the primary result.

Controls and early endings. Both Qwen models, Qwen3.6-35B-A3B and Qwen3.8-27B, score 100% in both arms on the two kinds of control. Gemma 4 26B-A4B scores 80% in both arms on the uncovered environment faults, and 100% against 80% on the code defects, where two MCP runs ended early after exhausting their reminders; they count as failures. Three Gemma 4 26B-A4B MCP runs in scope ended early in the same way. On the seven tasks both of Gemma 4 26B-A4B's arms solve, FixFirst saves no time.

Cloud model. DeepSeek V4.1 Flash ran through the provider's `deepseek-flash` alias on 9 October 2026 with thinking enabled, reasoning effort high and top-p 0.95; the provider controls the model behind the alias and the prompt cache. All 132 runs finished normally. Reasoning tokens were 176,051 without and 51,732 with FixFirst. Time and tokens fell on ten tasks and rose on eight (sign test for time p = 0.81); five tasks (ea4, eb3, ec4, ec5, ed2), picked after inspecting the results, account for most of the saving with a time ratio of 0.28. On the code-defect controls, total time and tokens with FixFirst were 1.84 and 1.77 times the baseline. Accounted at peak prices, the formal batch cost CNY 4.73. A separate checker confirmed the 132 commands, 1,320 requests and replies and 14,321 archived files.

Report compatibility across versions. No agent run was repeated on a later version. Replayed without a model, 0.8.0rc3 and rc4 return the reports of rc2 word for word for all 34 tasks of the two packs; rc5 differs in three, among them the SQLAlchemy task, whose first step now gives a change that can be followed.

Early runs on real projects. 78 runs over three models and three arms on two real projects, followed by six repeat runs, gave two successes and could not show a difference: one project's fault is application logic, the other states its cause in the error text, and the run conditions of that time cut replies short. These 84 development runs are not combined with the main result.

D.3 Real code defects

In default use on 0.8.0rc2, the registered matcher found the target test in 2 of 27 issues and a supplementary reading of parameterised tests in 3; eleven tornado runs pointed to an unrelated Cython example import, so the absence of an installation command does not mean the first advice was right. In the second held-out set one Sanic defect does not reproduce without network access; it stays in the denominator. Over the 75 held-out defects with the tests named, the first step named a code or test defect for 62 (18 of them marked unconfirmed), another cause for 4 (each unconfirmed), and for 9 the failure was not reached. Used as development data afterwards, the phase-two defects and the first two sets give, on 0.8.0rc5, 74 seen and 69 closed correctly with the tests named, and 25 and 8 in default use.

D.4 Diagnosis on generated cases

The historical evaluation of the earlier 44-feature tree reported a hybrid accuracy of 0.930 with a scenario and 1.000 with a template held out, and in its ablation 0.488 for rules without knowledge, 0.791 with knowledge, 0.828 after heuristics and 0.609 for the tree alone. These are kept as historical results and are not attributed to the current model. The macro averages in Table 20 are computed from the per-class counts in `experiments/diagnosis_baseline/generated-results-20261004/COMPARISON.json`.

D.5 Diagnosis on the sealed tasks

Labels. Each step of a task's reference repair was mapped to a class before the analysis was run. Installing a distribution that was not installed, other than the project itself, is a missing dependency. Changing the installed version of a distribution, or editing the project to replace or drop a library or standard-library API, keyword or module that the installed version no longer provides, is a version incompatibility. Installing the project itself or adding an import path to the test configuration is a local module problem; adding a configuration value the project needs to run is missing configuration; changing the project's own logic is a code defect. A task's label is the set of the classes of its steps, and FixFirst's outputs were not consulted.

Scoring and intervals. A failure counts as right when the predicted class is in its task's set; no prediction counts as wrong. The strict reading keeps the 12 tasks with one class and their 17 failures. Both were registered; the intervals were computed after the results. Tasks are the independent units, so the intervals in Section 7.2.2 are Wilson intervals over tasks; resampling tasks gives 0.56–0.97 for the tree's share of right failures.

The tree of the registered analysis. The registered analysis fitted its tree as the evaluation command does, on the repository's example dataset. That dataset holds the same 215 cases as the shipped tree's training table but without their source snapshots, so 20 of the 81 features take other values, and the fitted tree has 19 nodes instead of 13; alone it was right in 21 of 29 failures. The shipped tree, which a script rebuilds byte for byte from its table, was then scored with the same labels, checks and scoring, and this second reading was registered before it was run. The two trees differ on two failures, both missing distributions, which the shipped tree classifies correctly. Labels and first checks were not changed.

Table 30 The sealed tasks: classes of the reference repair, failures in the first check of 0.8.0rc2, and failures with a right class. The second letter of a task's name gives its kind: a dependency or version bound, b removed API, c configuration or layout, d combined faults, e uncovered environment fault and f code defect, the last two being controls. MD missing dependency, VI version incompatibility, LM local module, CM missing configuration, CD code defect.

Task	Classes	Failures	Full hybrid	Tree alone	Without knowledge graph
ea1-pkgres-removed	VI	1	1	1	0
ea2-missing-httpx-mock	MD	3	3	0	3
ea3-missing-pycryptodome	MD	1	1	1	1
ea4-py-spec-625	VI	1	1	0	0
eb1-sqlalchemy-databases	VI	1	1	1	1
eb2-scipy-kulsinski	VI	1	1	1	1
eb3-werkzeug-bind-arguments	VI	1	1	1	1
eb4-sklearn-multi-class	VI	4	4	4	0
ec1-old-pytest-pyproject	VI	1	1	0	0
ec2-lib-layout	LM	1	1	1	1
ec3-metadata-needs-install	LM	1	1	1	1
ec4-django-makefile	MD, CM	3	3	3	3
ec5-django-runtests	MD, CM	4	4	4	4
ed1-pptx-and-pinv2	MD, VI	2	2	2	2
ed2-toolchain-layered	VI	1	1	0	0
ed3-three-independent	MD, VI	1	1	1	1
ed4-layout-dependency-kwarg	MD, VI, LM	1	1	1	1
ed5-django-settings-and-signal	MD, VI, CM	1	1	1	0
ee1-addopts-missing-plugin	MD	0	–	–	–
ee2-xlrd-xlsx	MD, VI	2	0	0	0

Task	Classes	Failures	Full hybrid	Tree alone	Without knowledge graph
ef1-code-defect-easy	CD	2	2	2	2
ef2-code-defect-moderate	CD	4	4	4	4

D.6 Next-step ordering

On the five generated multi-fault projects, FixFirst, the order of the messages, a fixed order of categories and FixFirst with exact or without grouping all found the first issue in 5 of 5, made no wrong attempts and reached the goal in 5 of 5.

D.7 Participant study

Each participant completed both tasks in both conditions; no false completion claims or edited tests were recorded. Median confidence was 5 with FixFirst and 4.5 without; SUS scores ranged from 75 to 92.5 (standard deviation 7.0).

Table 31 Mean task time of each participant.

Participant	Mean seconds with FixFirst	Mean seconds without	Difference
P01	91.5	141.5	−50
P02	83	87	−4
P03	137	127	+10
P04	96.5	127.5	−31
P05	134	154	−20
P06	162.5	192.5	−30